Sistema de Gestión de Stock para Multiserv

Documentación Técnica del Sistema

Multiserv Stock

Versión 1.0

Autor del proyecto:

Enzo Falcón

Correo: enalfaga@gmail.com

GitHub: https://github.com/enzofalcon

Tecnologías utilizadas:

PHP 8 · MySQL 8 · Docker · Linux · HTML · CSS · JavaScript

GS Reparaciones & Multiserv.Uy

Montevideo, Uruguay – 2025

Índice de contenidos

Sistema de Gestión de Stock para Multiserv 1
Documentación Técnica del Sistema 1
Multiserv Stock Versión 1.0 1
Índice de contenidos 2
Introducción 3
Objetivos del sistema 4
- Objetivo general 4
- Objetivos específicos 4

Alcance del sistema 5
Requerimientos del sistema 6
- Requerimientos funcionales 6
- Requerimientos no funcionales 7

Casos de usos 9
Modelo de dominio 17
- Descripción de los conceptos del dominio 17
- Observaciones 20

Modelo de datos 24
- Modelo Entidad Relación 24
- Restricciones 25
- Modelo lógico de la base de datos 27
- Decisiones de diseño del modelo de datos 29

Diseño del sistema 32
- Diagrama de clases del modelo conceptual 33
- Diagrama de clases del diseño técnico (Arquitectura MVC) 34

Arquitectura general del sistema 38
Capa de presentación – Interfaz de usuario 39
- Flujo del usuario – Gestión de productos 39

Validación funcional de la interfaz 42
Autenticación y seguridad 46
- Inicialización del módulo de autenticación 46
- Consideraciones de seguridad 46

Dockerización 47
Limitaciones de la versión 53
Proyecciones a versiones futuras 54
Bibliografía 56
Anexo A – Manual de usuario 57
- Inicio de sesión 57
- Panel principal 58
- Gestión de productos 59
- Gestión de proveedores 63
- Gestión de sucursales 68
- Cierre de sesión 71

Anexo B - Modelo físico de la base de datos 72
Anexo C - Implementación de la API y capa de acceso a datos (DAO) 81
Anexo D - Diseño de Interfaz (UX / UI) 84
Anexo E - Despliegue 87
Anexo F – Repositorio del sistema 99
Anexo G – Glosario de términos 100

Introducción

El presente documento describe el análisis, diseño e implementación del sistema Multiserv Stock, desarrollado como solución para la gestión de inventario de la empresa GS Reparaciones & Multiserv.Uy.

El sistema surge a partir de la necesidad de contar con una herramienta que permita registrar, consultar y controlar la disponibilidad de productos distribuidos en distintas sucursales, asegurando consistencia de la información (la trazabilidad de movimientos queda prevista para futuras versiones) y consistencia de la información almacenada.

La documentación se organiza siguiendo un enfoque incremental, comenzando por el modelo conceptual del dominio, continuando con el modelo de datos y culminando con la descripción de la arquitectura e implementación técnica.

Este documento corresponde a la versión actual del sistema y refleja tanto las decisiones de modelado como las restricciones y reglas de negocio definidas durante el proceso de desarrollo.

El código fuente del sistema se encuentra disponible en el siguiente repositorio:

https://github.com/enzofalcon/multiserv-stock

Objetivos del sistema

Objetivo general

Desarrollar un sistema de gestión de stock que permita administrar productos, proveedores y sucursales, garantizando la consistencia de la información almacenada y sentando las bases para futuras funcionalidades de trazabilidad y la integridad de la información almacenada.

Objetivos específicos

- Modelar conceptualmente el dominio del problema utilizando UML.
- Diseñar un modelo de datos normalizado hasta Tercera Forma Normal (3FN).
- Implementar una base de datos relacional en MySQL 8.
- Desarrollar una API REST en PHP 8 para el acceso a datos.
- Implementar una interfaz web básica para interacción operativa.
- Incorporar mecanismos iniciales de autenticación de usuarios.
- Establecer una base arquitectónica que permita la evolución futura del sistema.

Alcance del sistema

El sistema Multiserv Stock se orienta exclusivamente a la gestión interna de inventario y control de disponibilidad de productos.

Incluye:

- Gestión de productos.
- Registro histórico de costos informados por proveedores.
- Gestión de disponibilidad por sucursal.
- Configuración limitada de sucursales.
- Implementación inicial de autenticación de usuarios.

No incluye:

- Gestión de ventas.
- Facturación.
- Control contable.
- Gestión de clientes.
- Automatización de reposición.
- Sistema de reportes avanzados.

El sistema está diseñado como herramienta interna de uso operativo, priorizando simplicidad, coherencia estructural y trazabilidad de información.

Requerimientos del sistema

Los requerimientos funcionales definen qué debe hacer el sistema, es decir, las funcionalidades, comportamientos y servicios que debe ofrecer a la empresa.
Los requerimientos no funcionales establecen cómo debe comportarse el sistema, incluyendo aspectos técnicos, de calidad, seguridad, rendimiento, arquitectura e infraestructura que condicionan su implementación.

Requerimientos funcionales

Los requerimientos funcionales del Sistema de Gestión de Stock para Multiserv se centran en la administración de inventario, productos, proveedores y movimientos asociados, garantizando la trazabilidad de la información y el control operativo del stock.

Gestión de productos

- Registrar productos.
- Visualizar listado de productos.
- Editar productos existentes (funcionalidad prevista para versiones futuras).
- Asociar productos a proveedores.
- Registrar costo de compra.
- Gestionar stock mínimo.
- Visualizar estado actual de stock.

Gestión de proveedores

- Registrar proveedores.
- Almacenar RUT y correo único.
- Registrar datos de contacto.
- Gestionar estado del proveedor.

Gestión de stock

- Registrar movimientos de stock (funcionalidad prevista para futuras versiones).
- Asociar movimientos a productos.
- Controlar que la cantidad no sea negativa.
- Visualizar productos cuyo stock actual se encuentre por debajo del stock mínimo definido.

Gestión de usuarios

- Registrar usuarios internos.

- Autenticar usuarios mediante credenciales.
- Administrar estado activo/inactivo.

Consulta y reportes

- Consultar stock por producto.
- Generar reportes básicos de inventario (aun no implementada en esta versión).

Requerimientos no funcionales

Los requerimientos no funcionales establecen las condiciones técnicas y de calidad bajo las cuales debe desarrollarse el sistema, asegurando seguridad, mantenibilidad, escalabilidad e integridad de los datos.

Estos requerimientos abarcan aspectos relacionados con la arquitectura del sistema, la seguridad informática, el uso de tecnologías específicas, las prácticas de desarrollo y el entorno de despliegue.

Arquitectura

- Implementar patrón MVC.
- Separación estricta entre frontend y backend.
- Comunicación mediante APIs REST.
- Aplicaciones independientes por módulo.

Seguridad

- Encriptación de contraseñas.
- Prevención de SQL Injection mediante consultas preparadas.
- Sanitización de entradas.
- No exponer datos sensibles en respuestas JSON.
- Estado de usuario.

Base de datos

- MySQL 8.
- Normalización hasta 3FN.
- Integridad referencial mediante claves foráneas.
- Uso de transacciones (commit/rollback).
- Uso del motor InnoDB.

Infraestructura

- Entorno Linux.
- Contenedores Docker.

- Código versionado en GitHub.
- Despliegue modular.

Calidad

- Pruebas funcionales.
- Pruebas de caja negra y valores límite.
- Documentación técnica completa.
- Desarrollo iterativo.

Requerimientos de calidad y gestión

- Documentación bajo estándares formales.
- Código organizado y mantenible.
- Modularidad y escalabilidad.
- Seguridad en todas las capas.
- Posibilidad de integración futura con sistemas existentes.

Casos de usos

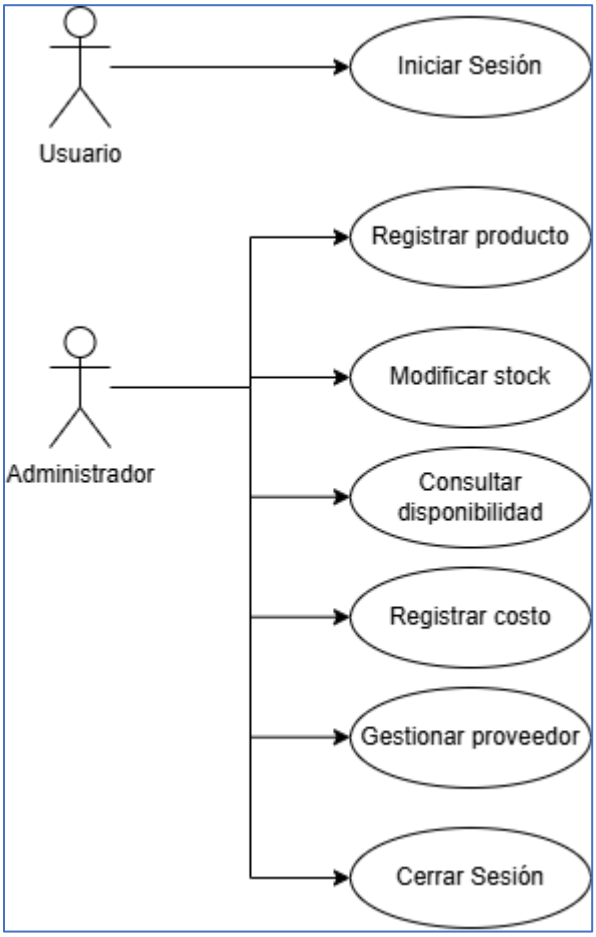
El presente apartado describe los casos de uso correspondientes a esta versión, documentando el comportamiento funcional actualmente implementado en el sistema.

La aplicación se encuentra desarrollada bajo una arquitectura API REST con capa DAO, utilizando PHP con manejo de sesión para autenticación. Todos los endpoints responden en formato JSON, y las vistas consumen dichos servicios mediante solicitudes HTTP.

En esta versión:

- Existe un único actor funcional: Usuario autenticado.
- La autenticación se gestiona mediante sesión PHP.
- La actualización de stock impacta directamente en la tabla disponibilidad.
- La entidad movimiento_stock existe en el modelo físico, pero no participa en la operativa actual.
- La baja de proveedor se realiza mediante cambio de estado (baja lógica).
- No se incorporan funcionalidades no implementadas en la versión estable 1.0.

Los casos de uso aquí detallados reflejan exclusivamente el comportamiento real del sistema en su versión entregable actual.



Autenticar usuario - CU-MS-001

Campo	Detalle
Dependencias	-
Descripción	El caso de uso comienza cuando un usuario desea acceder al sistema Multiserv Stock mediante sus credenciales. El sistema valida los datos ingresados contra la base de datos y, si son correctos, crea una sesión PHP activa que habilita el acceso a los endpoints protegidos del sistema.
Actor	Usuario
Precondición	• El usuario se encuentra registrado en el sistema. • No posee una sesión activa.
Secuencia principal	Paso 1: El usuario ingresa email y contraseña en el formulario de login. Paso 2: El frontend envía una solicitud HTTP POST al endpoint de autenticación. Paso 3: El sistema valida las credenciales consultando la base de datos mediante la capa DAO.Paso 4: Las credenciales son correctas. Paso 4.1: El sistema crea una sesión PHP asociada al usuario. Paso 4.2: El sistema responde en formato JSON indicando autenticación exitosa. Paso 5: Finaliza el caso de uso.
Postcondición	• Se crea una sesión PHP activa asociada al usuario. • El usuario puede acceder a las vistas y endpoints protegidos del sistema.
Secuencia alternativa 1	Paso 4A: Las credenciales son incorrectas.

	Paso 4A.1: El sistema responde en formato JSON indicando error de autenticación. Paso 4A.2: No se crea sesión. Paso 4A.3: Finaliza el caso de uso.
Secuencia alternativa 2	Paso 4B: El usuario se encuentra inactivo. Paso 4B.1: El sistema responde con mensaje de error en formato JSON. Paso 4B.2: No se crea sesión. Paso 4B.3: Finaliza el caso de uso.
Comentarios	• La autenticación se implementa mediante sesión PHP. • Los endpoints protegidos validan la existencia de sesión activa mediante middleware. • La respuesta del endpoint se realiza en formato JSON conforme a la arquitectura API REST del sistema.

Registrar producto – CU-MS-002

Campo	Detalle
Dependencias	Autenticar usuario – CU-MS-001
Descripción	El caso de uso comienza cuando un usuario autenticado desea registrar un nuevo producto en el sistema. El sistema recibe los datos ingresados, los valida y, si son correctos, crea un nuevo registro en la tabla `producto` a través de la capa DAO. El registro de proveedor y costo puede realizarse en operaciones posteriores independientes.
Actor	Administrador (Usuario autenticado)
Precondición	• El usuario posee una sesión PHP activa. • El usuario accede a la vista de gestión de productos.
Secuencia principal	Paso 1: El usuario selecciona la opción de registrar producto. Paso 2: El sistema muestra el formulario correspondiente. Paso 3: El usuario completa los datos requeridos y confirma el envío. Paso 4: El frontend envía una solicitud HTTP POST al endpoint de productos. Paso 5: El sistema valida que la descripción haya sido enviada y no esté vacía. Paso 6: Los datos son correctos. Paso 6.1: Se inserta el nuevo registro en la tabla producto mediante la capa DAO. Paso 6.2: El sistema responde en formato JSON indicando éxito en la operación. Paso 7: Finaliza el caso de uso.
Postcondición	• Se crea un nuevo registro en la tabla producto. • El producto queda disponible para su consulta y gestión posterior.
Secuencia alternativa 1	Paso 6A: Los datos ingresados son inválidos o incompletos. Paso 6A.1: El sistema responde en formato JSON indicando error de validación. Paso 6A.2: No se crea el registro. Paso 6A.3: Finaliza el caso de uso.

Secuencia alternativa 2	Paso 6B: Se produce un error interno en la base de datos. Paso 6B.1: El sistema responde con error en formato JSON. Paso 6B.2: No se crea el registro.Paso 6B.3: Finaliza el caso de uso.
Comentarios	• La persistencia se realiza exclusivamente mediante la capa DAO. • El endpoint responde en formato JSON conforme a la arquitectura REST. • No se generan registros automáticos en otras tablas como parte de esta operación en versión 1.0.

Modificar stock – CU-MS-003

Campo	Detalle
Dependencias	Autenticar usuario – CU-MS-001
Descripción	El caso de uso comienza cuando el Administrador desea actualizar la cantidad disponible de un producto en una sucursal determinada. El sistema recibe la nueva cantidad y actualiza directamente el registro correspondiente en la tabla disponibilidad.
Actor	Administrador (Usuario autenticado)
Precondición	• El Administrador posee sesión activa. • Existe un registro previo en la tabla disponibilidad para el producto y la sucursal seleccionados.
Secuencia principal	Paso 1: El Administrador accede a la gestión de stock. Paso 2: Selecciona el producto y la sucursal correspondiente. Paso 3: Ingresa la nueva cantidad disponible. Paso 4: El frontend envía una solicitud HTTP al endpoint correspondiente (PUT o método definido en la API). Paso 5: El sistema valida los datos recibidos. Paso 6: El registro existe y los datos son válidos. Paso 6.1: El sistema actualiza directamente el campo de cantidad en la tabla disponibilidad mediante la capa DAO. Paso 6.2: El sistema responde en formato JSON indicando éxito en la operación. Paso 7: Finaliza el caso de uso.
Postcondición	• La cantidad disponible del producto en la sucursal queda actualizada en la tabla disponibilidad.
Secuencia alternativa 1	Paso 6A: No existe registro de disponibilidad para el producto y sucursal. Paso 6A.1: El sistema responde en formato JSON indicando error. Paso 6A.2: No se realiza actualización.
Secuencia alternativa 2	Paso 5B: Los datos enviados son inválidos (por ejemplo, cantidad no numérica). Paso 5B.1: El sistema responde con error en formato JSON. Paso 5B.2: No se actualiza el registro.
Comentarios	• En la versión 1.0 la modificación de stock impacta directamente en la tabla disponibilidad. • La entidad movimiento_stock existe en el modelo físico, pero no participa en la operativa actual. • No se registra historial de cambios en esta versión.

Consultar disponibilidad – CU-MS-004

Campo	Detalle
Dependencias	Autenticar usuario – CU-MS-001
Descripción	El caso de uso comienza cuando el Administrador desea visualizar la cantidad disponible de los productos registrados en el sistema. El sistema consulta la base de datos y devuelve la información actualizada en formato JSON.
Actor	Administrador (Usuario autenticado)
Precondición	• El Administrador posee sesión activa. • Existen productos registrados en el sistema.
Secuencia principal	Paso 1: El Administrador accede al módulo de consulta de disponibilidad. Paso 2: El frontend envía una solicitud HTTP GET al endpoint correspondiente. Paso 3: El sistema consulta la información de productos y su stock total mediante la capa DAO. Paso 4: El sistema construye la respuesta con los datos obtenidos. Paso 5: El sistema responde en formato JSON con el listado de productos y su disponibilidad. Paso 6: Finaliza el caso de uso.
Postcondición	• No se modifican datos en la base de datos. • El Administrador visualiza la disponibilidad actualizada de los productos.
Secuencia alternativa 1	Paso 3A: No existen productos registrados. Paso 3A.1: El sistema responde con una lista vacía en formato JSON. Paso 3A.2: Finaliza el caso de uso.
Secuencia alternativa 2	Paso X: Se produce un error interno del servidor. Paso X.1: El sistema responde con código HTTP 500 y mensaje JSON de error. Paso X.2: Finaliza el caso de uso.
Comentarios	• La consulta se realiza mediante arquitectura REST. • La información de stock se obtiene a partir de la tabla disponibilidad. • No se consulta ni registra información en movimiento_stock en la versión 1.0.

Registrar costo – CU-MS-005

Campo	Detalle
Dependencias	Autenticar usuario – CU-MS-001
Descripción	El caso de uso comienza cuando el Administrador desea registrar un nuevo costo para un producto existente. El sistema recibe los datos enviados y los persiste en la tabla correspondiente a costos mediante la capa DAO.
Actor	Administrador (Usuario autenticado)
Precondición	• El Administrador posee sesión activa. • El producto al cual se le registrará el costo existe en el sistema.
Secuencia principal	Paso 1: El Administrador accede a la opción de registrar costo. Paso 2: Selecciona el producto correspondiente. Paso 3: Ingresa el valor del costo. Paso 4: El frontend envía una solicitud HTTP al endpoint correspondiente.

	Paso 5: El sistema valida los datos recibidos. Paso 6: Los datos son correctos. Paso 6.1: El sistema inserta el nuevo registro en la tabla de costos mediante la capa DAO. Paso 6.2: El sistema responde en formato JSON indicando éxito en la operación. Paso 7: Finaliza el caso de uso.
Postcondición	• Se crea un nuevo registro de costo asociado al producto. • El nuevo costo queda disponible para consultas posteriores.
Secuencia alternativa 1	Paso 5A: El producto no existe. Paso 5A.1: El sistema responde con error en formato JSON. Paso 5A.2: No se registra el costo. Paso 5A.3: Finaliza el caso de uso.
Secuencia alternativa 2	Paso 5B: El valor de costo es inválido. Paso 5B.1: El sistema responde con error en formato JSON. Paso 5B.2: No se registra el costo. Paso 5B.3: Finaliza el caso de uso.
Comentarios	• La operación se realiza bajo arquitectura REST. • La persistencia se implementa mediante la capa DAO. • En la versión 1.0 no se elimina historial de costos previamente registrados.

Gestionar proveedor – CU-MS-006

Campo	Detalle
Dependencias	Autenticar usuario – CU-MS-001
Descripción	El caso de uso comienza cuando el Administrador desea gestionar los proveedores del sistema. La gestión incluye el registro, modificación y desactivación (baja lógica) de proveedores. La eliminación física no forma parte de la operativa de la versión 1.0.
Actor	Administrador (Usuario autenticado)
Precondición	• El Administrador posee sesión activa. • El módulo de proveedores se encuentra accesible.
Secuencia principal (Alta / Modificación)	Paso 1: El Administrador accede al módulo de proveedores. Paso 2: Selecciona la opción de registrar o editar proveedor. Paso 3: Ingresa o modifica los datos correspondientes. Paso 4: El frontend envía la solicitud HTTP al endpoint correspondiente. Paso 5: El sistema valida los datos recibidos. Paso 6: Los datos son correctos. Paso 6.1: El sistema inserta o actualiza el registro en la tabla proveedor mediante la capa DAO. Paso 6.2: El sistema responde en formato JSON indicando éxito. Paso 7: Finaliza el caso de uso.
Secuencia principal (Baja lógica)	Paso 1: El Administrador selecciona un proveedor existente. Paso 2: Ejecuta la opción de desactivar proveedor. Paso 3: El frontend envía una solicitud HTTP DELETE al endpoint correspondiente.

	Paso 4: El sistema actualiza el campo de estado del proveedor a inactivo. Paso 5: El sistema responde en formato JSON indicando éxito. Paso 6: Finaliza el caso de uso.
Postcondición	• El proveedor queda registrado, actualizado o marcado como inactivo según la operación realizada. • En caso de baja lógica, el registro permanece en la base de datos con estado inactivo.
Secuencia alternativa 1	Paso 5A: Los datos enviados son inválidos. Paso 5A.1: El sistema responde con error en formato JSON. Paso 5A.2: No se realiza la operación. Paso 5A.3: Finaliza el caso de uso.
Secuencia alternativa 2	Paso X: Se produce un error interno del servidor. Paso X.1: El sistema responde con código HTTP 500 y mensaje JSON de error. Paso X.2: Finaliza el caso de uso.
Comentarios	• La baja de proveedor se realiza mediante cambio de estado (baja lógica). • No se elimina físicamente el registro en la versión 1.0. • La persistencia se implementa mediante arquitectura REST + capa DAO. • Las respuestas del sistema se entregan en formato JSON.

Cerrar sesión – CU-MS-007

Campo	Detalle
Dependencias	Autenticar usuario – CU-MS-001
Descripción	El caso de uso comienza cuando el Administrador desea finalizar su sesión activa en el sistema. El sistema invalida la sesión PHP y revoca el acceso a los endpoints protegidos.
Actor	Administrador (Usuario autenticado)
Precondición	• El Administrador posee una sesión PHP activa.
Secuencia principal	Paso 1: El Administrador selecciona la opción cerrar sesión. Paso 2: El frontend envía una solicitud HTTP al endpoint correspondiente. Paso 3: El sistema ejecuta la destrucción de la sesión PHP activa. Paso 4: El sistema responde en formato JSON indicando éxito. Paso 5: Finaliza el caso de uso.
Postcondición	• La sesión PHP queda invalidada. • El usuario no puede acceder a los endpoints protegidos sin autenticarse nuevamente.
Secuencia alternativa 1	Paso 3A: No existe sesión activa. Paso 3A.1: El sistema responde en formato JSON indicando estado inválido o sesión inexistente. Paso 3A.2: Finaliza el caso de uso.
Secuencia alternativa 2	Paso X: Se produce un error interno del servidor. Paso X.1: El sistema responde con código HTTP 500 y mensaje JSON de error. Paso X.2: Finaliza el caso de uso.

Comentarios	• La autenticación se basa en manejo de sesión PHP. • La invalidación de sesión impide el acceso a los endpoints protegidos mediante el middleware de autenticación. • La respuesta del sistema se entrega en formato JSON conforme a la arquitectura REST.

Modelo de dominio

En la presente sección se muestra el modelo de dominio del proyecto.

"Un modelo de dominio es una representación de las clases conceptuales del mundo real, no de componentes software."

El diagrama que modela el dominio del sistema Multiserv Stock se realiza utilizando el lenguaje UML.

Se identifican en él:

- Conceptos, con sus respectivos atributos.
- Las asociaciones entre los distintos conceptos, identificados con un nombre y direccionalidad.
- La multiplicidad de las asociaciones.
- Las entidades del tipo asociativos.

Descripción de los conceptos del dominio

Previo a la presentación del diagrama de dominio, se describen a continuación los principales conceptos identificados en el sistema Multiserv Stock, de acuerdo con el modelo propuesto.

Producto

Representa un artículo físico gestionado por el sistema.

Cada producto cuenta con un identificador interno asignado por el sistema, una descripción y un stock mínimo definido como criterio preventivo de reposición.

El costo de adquisición no se modela como atributo directo del producto, sino mediante registros históricos asociados a proveedores.

El producto no contempla precios de venta, ya que el sistema se orienta exclusivamente al control de stock.

Sucursal

Representa una ubicación física de la empresa en la cual se almacenan productos.

Cada sucursal se identifica de manera única mediante un número interno y dispone de información de contacto, así como de una dirección asociada que permite su localización física.

El stock disponible se gestiona de forma diferenciada para cada sucursal.

Cada sucursal posee un estado (activa o inactiva), el cual determina si puede registrar movimientos de stock.

Proveedor

Representa a la entidad externa que informa los costos de adquisición de productos.

Se registran sus datos identificatorios y de contacto necesarios para su gestión dentro del sistema.

Un proveedor puede informar costos para uno o varios productos. Asimismo, cada proveedor puede registrar información de contacto asociada, incluyendo su dirección.

La condición impositiva y los valores económicos asociados a los productos se registran de manera independiente para cada costo informado.

Dirección

Representa la información de localización física asociada a una sucursal o a un proveedor.

Incluye datos como departamento, localidad y referencias de calle, permitiendo identificar direcciones locales o del exterior según corresponda.

Una dirección puede estar asociada a una única sucursal o a un único proveedor, y su existencia es independiente del registro de productos o stock.

Disponibilidad

Representa la existencia de un producto en una sucursal determinada.

Permite asociar un producto con una ubicación específica y registrar la cantidad disponible en dicho lugar.

La cantidad registrada es independiente tanto del producto como de la sucursal considerados de forma aislada, ya que surge de la relación entre ambos.

RegistroCosto

Representa el costo de adquisición informado por un proveedor para un producto en un momento determinado.

Cada registro de costo se asocia a un único producto y a un único proveedor, e incluye el valor del costo, la fecha y hora en que fue informado y la condición impositiva correspondiente (por ejemplo, si el costo incluye IVA o no).

Los registros de costo tienen carácter informativo e histórico, permitiendo conservar variaciones de valores a lo largo del tiempo, sin generar cálculos automáticos ni interpretaciones comerciales dentro del sistema.

MovimientoStock

En la versión actual del sistema Multiserv Stock, las modificaciones de inventario se realizan directamente sobre la entidad Disponibilidad, la cual mantiene el estado persistido de la

cantidad disponible para cada combinación de producto y sucursal. Por este motivo, el sistema actualmente no registra un historial de movimientos de stock.

No obstante, el modelo de datos incluye la tabla movimiento_stock, diseñada para permitir en futuras versiones el registro histórico de las variaciones de inventario. Esta entidad permitiría asociar cada modificación de stock a un Producto, una Sucursal y un Usuario, registrando además el tipo de movimiento, la cantidad modificada, la fecha y hora de la operación y, opcionalmente, un motivo descriptivo.

El objetivo de esta estructura es proporcionar trazabilidad sobre las variaciones de inventario, permitiendo reconstruir el historial de cambios de stock y facilitar tareas de auditoría o análisis operativo.

Actualmente, esta entidad se encuentra definida en el modelo físico de la base de datos, pero no forma parte de la lógica operativa del sistema, quedando prevista como una posible mejora para futuras evoluciones del proyecto.

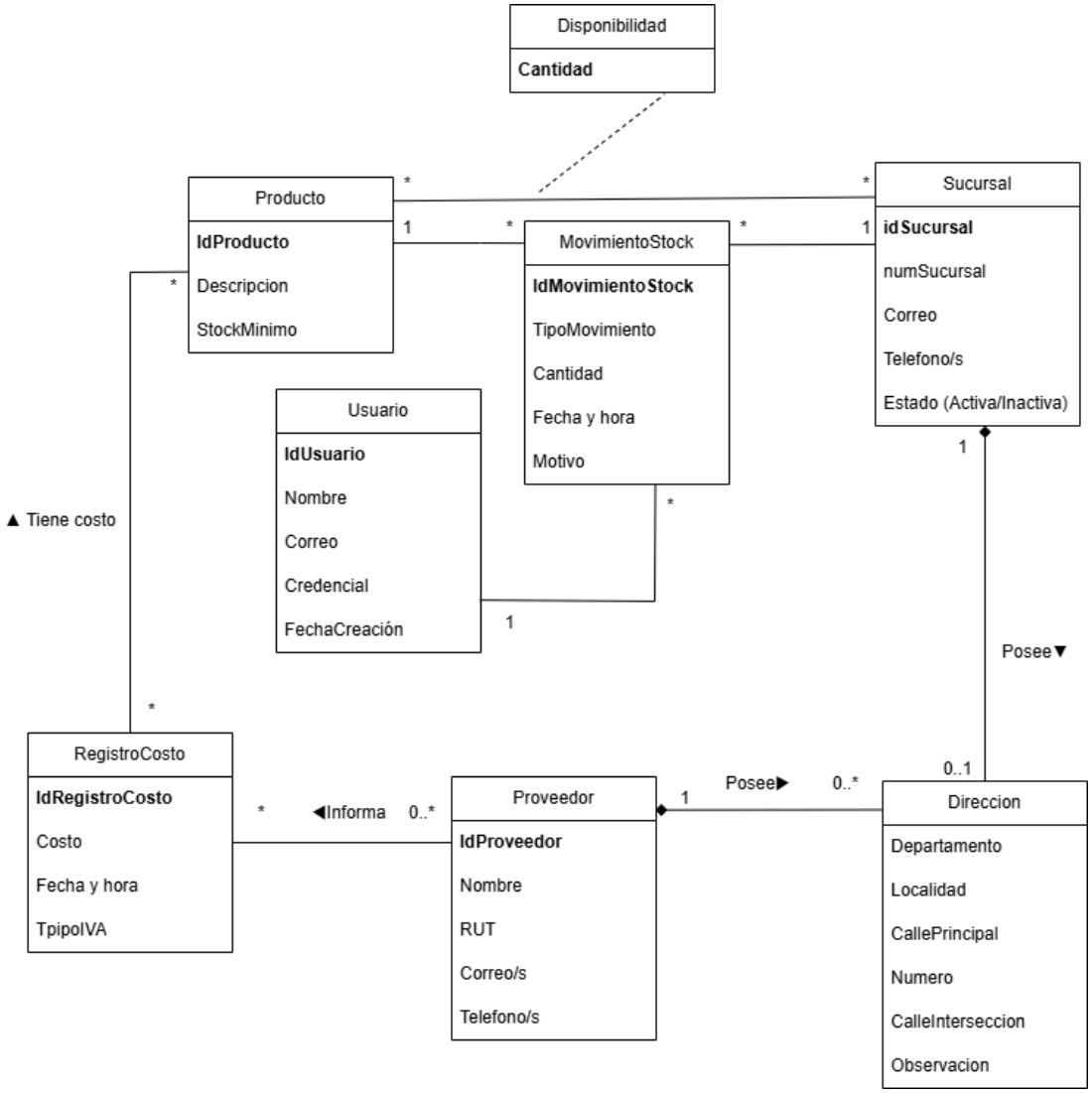
Usuario

Representa a la persona autorizada a operar el sistema.

Cada usuario cuenta con un identificador interno, nombre, correo electrónico y una credencial utilizada para autenticación.

Los usuarios registran movimientos de stock, permitiendo mantener trazabilidad sobre las acciones realizadas en el sistema.

A continuación, se presenta el diagrama de dominio correspondiente al sistema Multiserv Stock.



Observaciones

En esta sección se expresan aclaraciones necesarias para esclarecer y complementar el Modelo de Dominio, explicitando supuestos y reglas conceptuales que no se representan de forma gráfica.

Restricciones

Las siguientes restricciones corresponden a decisiones de modelado del dominio

<u>Unicidad de atributos</u>

En el Modelo de Dominio se definen ciertos atributos cuya finalidad es identificar de manera unívoca a las entidades principales del sistema.

Cada Producto, Sucursal, Proveedor, RegistroCosto y Movimiento de Stock cuenta con un identificador interno asignado por el sistema, el cual permite distinguir de forma única cada

instancia dentro del dominio. Dichos identificadores no representan aún claves técnicas de base de datos, sino identificadores conceptuales propios del dominio.

Asimismo, se asume la unicidad de ciertos atributos relevantes del negocio:

- El RUT del proveedor no puede repetirse dentro del sistema (cuando corresponda aplicarlo).

Dominio de atributos

Los atributos definidos en el Modelo de Dominio representan información propia del mundo real y se restringen a valores coherentes con su significado.

- Los atributos identificadores (IdProducto, IdSucursal, IdProveedor, IdRegistroCosto, IdMovimientoStock) representan valores numéricos asignados por el sistema.
- Los atributos descriptivos, como nombre, descripción, departamento, localidad o calles, corresponden a valores textuales.
- El atributo Cantidad, presente en la entidad Disponibilidad, representa un valor numérico entero no negativo y refleja el estado actual de la existencia de un producto en una sucursal determinada.
- El atributo Cantidad, presente en la entidad Movimiento de Stock, representa la variación producida sobre la disponibilidad como resultado de una operación de ingreso, egreso, ajuste o carga inicial.
- El atributo Costo (en RegistroCosto) representa un valor numérico positivo, asociado al costo de adquisición informado para un producto.
- PorcentajeIVA (valor porcentual aplicado) representa una condición impositiva asociada al costo informado.
- El atributo TipoMovimiento identifica la naturaleza del movimiento de stock realizado (por ejemplo, carga inicial, ingreso, egreso o ajuste).
- El atributo StockMinimo, presente en la entidad Producto, representa la cantidad mínima recomendada como criterio preventivo de reposición.

No se especifican tipos de datos técnicos en esta etapa, ya que el modelo corresponde al nivel de dominio y no a la implementación.

Atributos calculados

El Modelo de Dominio no define atributos calculados de manera explícita, dado que su objetivo es representar conceptos y relaciones del mundo real.

No obstante, se reconoce que ciertos valores pueden derivarse a partir de la información existente en el sistema. Por ejemplo, el stock total de un producto puede obtenerse como la suma de las cantidades registradas en las distintas instancias de Disponibilidad asociadas a dicho producto.

Las condiciones impositivas (por ejemplo, inclusión o no de IVA) no se consideran atributos calculados, sino reglas propias del dominio, y se modelan mediante atributos o reglas de negocio (como TipoIVA en RegistroCosto).

El atributo CostoActual, asociado conceptualmente a Producto, se considera un atributo derivado que puede obtenerse a partir del RegistroCosto más reciente informado por proveedores. En la versión actual del sistema, este valor no se almacena como dato independiente ni se muestra en las interfaces de usuario. Se mantiene únicamente como parte del modelo conceptual, ya que podría derivarse de otros datos existentes y utilizarse en futuras funcionalidades de análisis o generación de reportes.

<u>Reglas de negocio</u>

El Modelo de Dominio refleja las siguientes reglas de negocio fundamentales del sistema:

- Un Producto puede estar disponible en una o varias Sucursales, y la cantidad disponible depende de la relación entre Producto y Sucursal, representada mediante Disponibilidad.
- La cantidad disponible se actualiza directamente sobre la entidad Disponibilidad.
- El stock mínimo no altera automáticamente la disponibilidad, pero funciona como referencia para la toma de decisiones operativas.
- Un Movimiento de Stock se asocia a un único Producto y a una única Sucursal.
- Una Sucursal solo puede registrar movimientos de stock cuando se encuentra en estado activa.
- Una Sucursal puede contar con cero o una Dirección asociada, según la información disponible al momento del registro.
- Un Proveedor puede no tener direcciones registradas o puede contar con una dirección.
- Un Proveedor puede informar uno o varios costos para productos (RegistroCosto), y un Producto puede tener cero o más registros de costo informados por proveedores a lo largo del tiempo.
- La cantidad de un producto no es una propiedad del producto en sí, sino del contexto en el que se encuentra, determinado por la sucursal y los movimientos de stock asociados.

- La utilización obligatoria de MovimientoStock para registrar cada variación queda prevista para versiones futuras del sistema.
- Todo Movimiento de Stock debe estar asociado a un Usuario responsable que haya realizado la operación.

Modelo de datos

Siguiendo el enfoque orientado a objetos propuesto por Larman, el análisis del problema se realizó mediante un modelo de dominio UML, el cual permitió identificar los conceptos principales del sistema y sus relaciones.

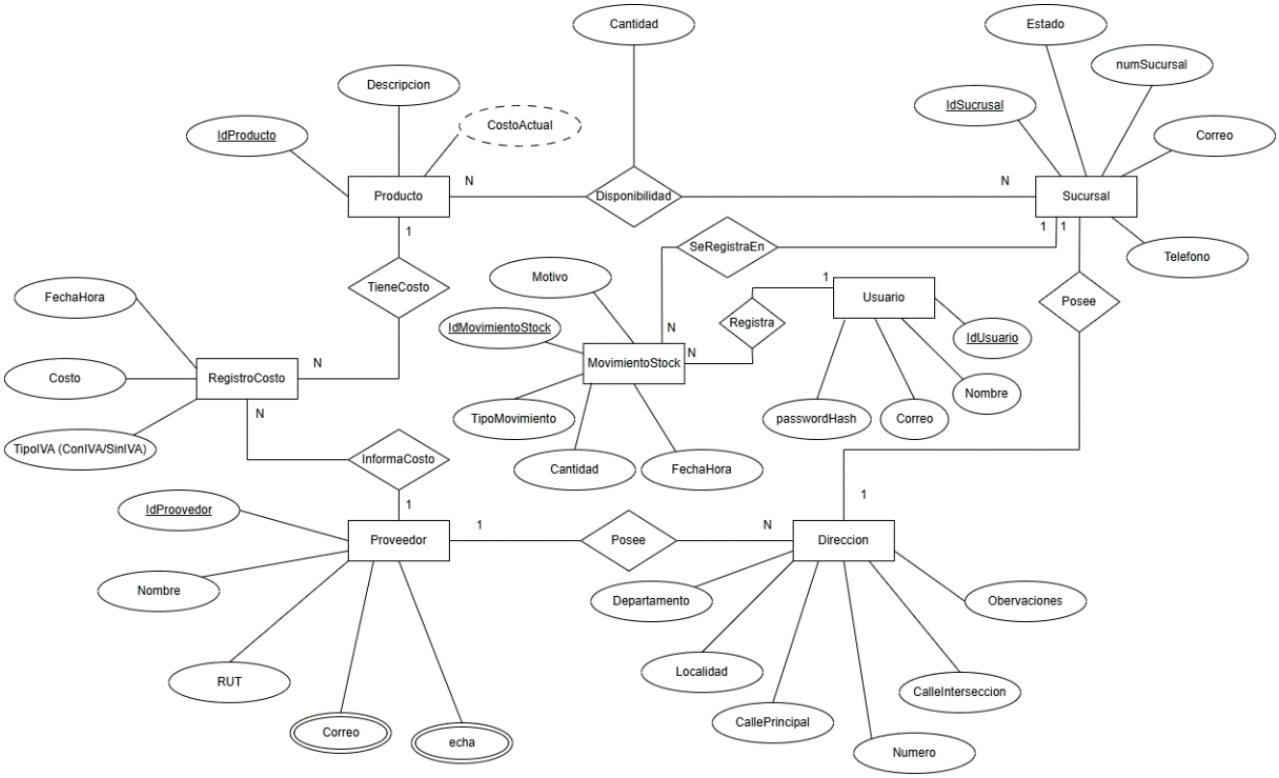
En una etapa intermedia entre el análisis y el diseño lógico, se elaboró el Modelo Entidad Relación (MER) como modelo conceptual de datos.

El MER permitió identificar y estructurar la información que el sistema debe persistir, definiendo entidades, atributos y relaciones, así como sus cardinalidades y restricciones conceptuales, sin introducir aún detalles propios de la implementación relacional.

Este modelo se construyó a partir de los conceptos identificados en el modelo de dominio, incorporando además entidades que permiten representar información histórica y de trazabilidad, como los RegistrosCosto y los MovimientosStock, necesarios para conservar la evolución temporal de los datos.

El MER constituye, de este modo, la base conceptual para la posterior elaboración del Diagrama Entidad Relación lógico (DER), a partir del cual se definirá el esquema relacional de la base de datos.

Modelo Entidad Relación



Restricciones

Con el objetivo de complementar el Modelo Entidad Relación, se identificaron las siguientes restricciones del sistema, clasificadas según su naturaleza.

Restricciones estructurales

Incluyen las restricciones que pueden ser expresadas directamente a través de la estructura del modelo.

- Cada Producto se identifica de forma única mediante su identificador.
- Cada Proveedor se identifica de forma única mediante su identificador.
- Cada Sucursal se identifica de forma única mediante su identificador.
- Cada MovimientoStock se identifica de forma única mediante su identificador.
- Todo RegistroCosto debe estar asociado a un único Producto.
- Todo RegistroCosto debe estar asociado a un único Proveedor.
- Un Producto puede tener cero o muchos RegistrosCosto.
- Un Proveedor puede informar cero o muchos RegistrosCosto.
- La Disponibilidad se define como la relación entre un Producto y una Sucursal, representando la cantidad existente de dicho producto en esa ubicación.
- Un Producto puede tener cero o muchas instancias de Disponibilidad.
- Una Sucursal puede tener cero o muchas instancias de Disponibilidad.
- Un Proveedor puede tener cero o varias Direcciones registradas.
- Una Sucursal puede tener cero o una Dirección asociada.
- Cada Usuario se identifica de forma única mediante su identificador.
- Todo MovimientoStock debe estar asociado a un único Usuario.

Restricciones no estructurales

Corresponden a reglas del sistema que no pueden representarse gráficamente en el MER y se expresan de forma textual. Estas restricciones regulan el comportamiento global de los datos y deben cumplirse para garantizar la consistencia del sistema.

- El CostoActual de un producto se obtiene a partir del RegistroCosto más reciente asociado a dicho producto ordenado por fechaHora y, en caso de igualdad, por identificador.
- Cada RegistroCosto debe indicar si el costo informado corresponde a un monto con IVA o sin IVA.

- Pueden existir múltiples RegistrosCosto para un mismo producto y proveedor, incluso con la misma fecha y hora, ya que los registros tienen carácter histórico e informativo.
- La cantidad disponible de un producto en una sucursal no puede ser negativa.
- El sistema no gestiona precios de venta; los costos registrados corresponden exclusivamente al costo de adquisición del producto.

Restricciones de estado y transición

Definen condiciones sobre los estados válidos del sistema y las transiciones permitidas entre dichos estados, en función del tiempo o de eventos específicos.

- Un Producto puede existir en el sistema sin registros de costo asociados.
- El CostoActual de un producto solo puede calcularse cuando existe al menos un RegistroCosto asociado.
- La cantidad disponible se modifica directamente sobre la entidad Disponibilidad. La utilización obligatoria de MovimientoStock para registrar cada variación queda prevista para versiones futuras.
- Un MovimientoStock debe estar asociado a un único Producto y a una única Sucursal.
- Un RegistroCosto, una vez creado, no puede ser modificado; ante un cambio de costo se debe generar un nuevo registro.
- La fecha y hora de un RegistroCosto no puede ser posterior a la fecha y hora actual del sistema.
- La eliminación de un Producto solo es posible si no existen registros de Disponibilidad ni registros históricos asociados, ni RegistrosCosto asociados. La restricción se implementa mediante integridad referencial en la base de datos.
- La entidad MovimientoStock se encuentra modelada en la base de datos para evolución futura del sistema.

Modelo lógico de la base de datos

En esta etapa se define el modelo lógico de la base de datos, a partir del modelo conceptual desarrollado previamente.

El objetivo es establecer la estructura de tablas, así como las claves primarias y claves foráneas necesarias para representar la información del sistema de forma consistente y preparada para su implementación en un gestor de bases de datos relacional.

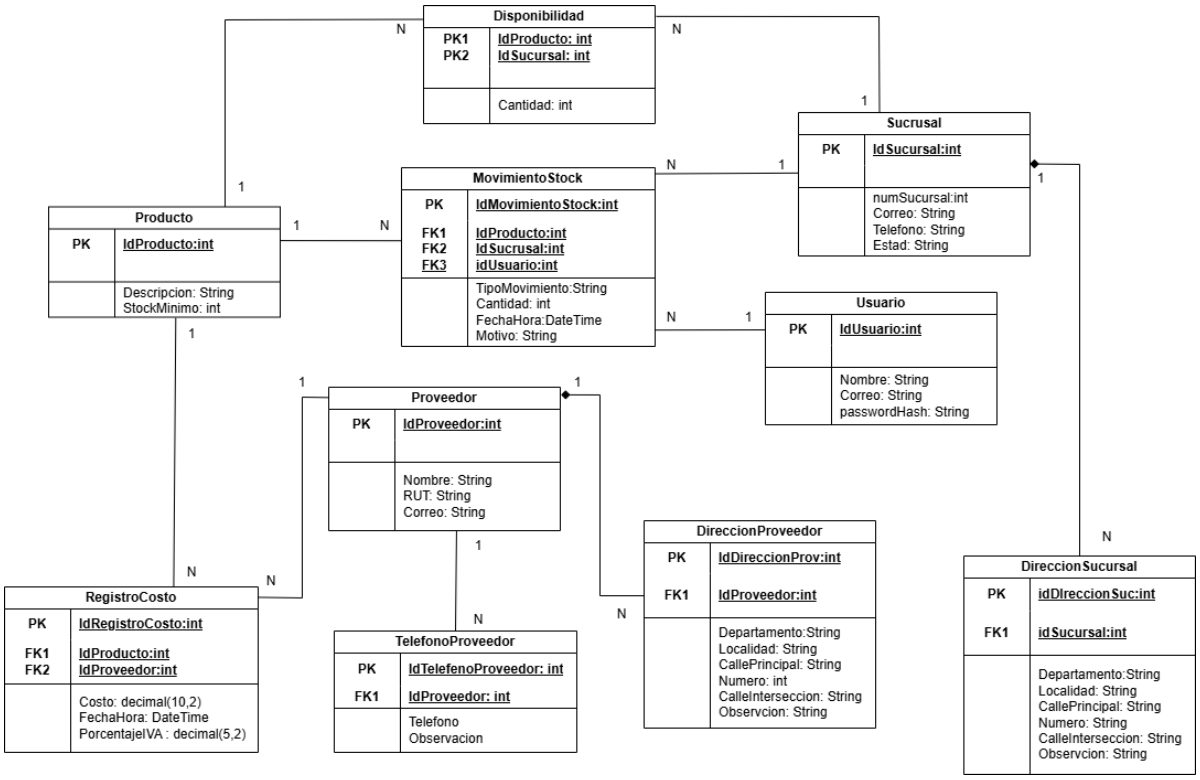
El modelo fue normalizado hasta Tercera Forma Normal (3FN), evitando redundancias y dependencias incorrectas entre atributos. Las relaciones de tipo muchos a muchos identificadas en el Modelo Entidad–Relación fueron resueltas mediante tablas intermedias, y aquellos datos que requieren historial o multiplicidad fueron modelados en entidades independientes, asegurando la integridad y trazabilidad de la información.

A partir del análisis del dominio y del alcance definido para el sistema, se adoptaron las siguientes decisiones de modelado lógico:

- Un producto puede estar disponible en una o varias sucursales, y la cantidad correspondiente se gestiona de forma independiente para cada una de ellas.
- El stock se representa mediante una tabla de Disponibilidad, que relaciona productos y sucursales y almacena el estado actual del inventario, evitando la duplicación de registros.
- Las variaciones de stock se registran en una entidad independiente (MovimientoStock), permitiendo conservar el historial de modificaciones sin afectar directamente el estado almacenado.
- Un producto puede contar con múltiples registros de costo asociados a distintos proveedores y a diferentes momentos en el tiempo, manteniendo un historial de costos informado.
- El porcentaje de IVA se almacena como un valor numérico asociado a cada registro de costo, permitiendo distinguir costos con y sin impuestos incluidos.
- Cada proveedor posee un único correo electrónico, considerado como canal formal de comunicación administrativa.
- Un proveedor puede registrar uno o más teléfonos, permitiendo identificar distintos contactos o usos.
- Cada sucursal cuenta con un único correo electrónico y un único teléfono de referencia.

- Las direcciones de proveedores y sucursales se modelan en entidades separadas, evitando ambigüedades y dependencias innecesarias dentro del modelo.

El modelo lógico definido constituye la base directa para la implementación de la base de datos, asegurando la coherencia entre la estructura de datos y las reglas establecidas para el funcionamiento del sistema, así como la correcta evolución hacia etapas posteriores de desarrollo.



Decisiones de diseño del modelo de datos

RegistroCosto como entidad histórica

Decisión

Se modela la variación de costos mediante la entidad *RegistroCosto*, permitiendo múltiples registros para un mismo producto y proveedor.

Justificación

El costo de un producto puede variar en el tiempo debido a cambios de proveedor, condiciones comerciales o impuestos. Mantener el historial evita la pérdida de información y permite análisis retrospectivos.

Regla de diseño

Los registros de costo no se modifican ni se eliminan; ante un nuevo costo, se registra una nueva instancia con su fecha y hora.

Separación de direcciones por entidad

Decisión

Las direcciones se modelan en entidades independientes (*DireccionProveedor* y *DireccionSucursal*), vinculadas mediante claves foráneas a sus respectivas entidades principales.

Justificación

Esta separación evita ambigüedades semánticas, reduce valores nulos y mantiene el modelo normalizado, facilitando futuras extensiones del sistema.

Decisión sobre sucursales

En el contexto del sistema Multiserv Stock, la definición de las sucursales responde a una decisión organizacional de la empresa, ya que representa la estructura física o administrativa donde se gestiona el inventario.

Por este motivo, las sucursales se consideran datos de configuración del sistema. En la versión actual, las sucursales pueden ser registradas y gestionadas desde la interfaz del sistema, de forma similar a la gestión de proveedores u otras entidades administrativas.

Esta decisión permite mantener una estructura flexible, en la que la empresa puede adaptar el sistema a cambios en su organización (por ejemplo, apertura de nuevas sucursales) sin requerir modificaciones técnicas en la base de datos.

A nivel operativo, el sistema utiliza la información de las sucursales para mostrar la disponibilidad de productos por sucursal, permitiendo visualizar el stock asociado a cada ubicación dentro del listado de productos.

Decisión sobre MovimientoStock como variaciones de stock

Decisión

Se modelan las variaciones de stock mediante la entidad **MovimientoStock**, la misma permitiría registrar múltiples movimientos asociados a un mismo producto y a una misma sucursal a lo largo del tiempo.

Justificación

La cantidad disponible de un producto puede cambiar por distintos motivos, como ingresos de mercadería, egresos por ventas, ajustes de inventario o cargas iniciales. Modelar estas variaciones como registros independientes permite conservar un historial completo de los cambios realizados, facilitando la trazabilidad del stock y el análisis retrospectivo de su evolución, sin perder información relevante.

Separar las variaciones del estado actual evita sobrescribir datos y permite reconstruir la historia del inventario en caso de auditorías, controles internos o ampliaciones futuras del sistema.

Regla de diseño

Los registros de MovimientoStock tienen carácter histórico e inalterable. Una vez registrado un movimiento, este no se modifica ni se elimina; cualquier cambio en la cantidad disponible debe registrarse mediante una nueva instancia de movimiento, indicando su tipo, cantidad y fecha y hora correspondientes.

Esta regla aplica como diseño previsto. En la versión 1.0, las variaciones de stock se actualizan directamente sobre Disponibilidad.

Separación entre estado y variaciones de stock

Decisión

Se separa el estado actual del stock (Disponibilidad) de las variaciones históricas (MovimientoStock).

Justificación

Esta separación evitará dependencias circulares, reduce redundancias y permite consultar eficientemente el stock vigente, manteniendo la posibilidad de análisis histórico.

Gestión de credenciales de usuario

Decisión

Las credenciales de los usuarios no se almacenan en texto plano. El sistema utiliza un mecanismo de almacenamiento mediante hash criptográfico (passwordHash).

El sistema implementa almacenamiento seguro de credenciales mediante hash criptográfico utilizando las funciones nativas password_hash() y password_verify() de PHP, evitando el almacenamiento de contraseñas en texto plano y cumpliendo buenas prácticas de seguridad.

Justificación

El almacenamiento de contraseñas en formato hash evita la exposición directa de credenciales en caso de acceso indebido a la base de datos, cumpliendo buenas prácticas de seguridad y reduciendo riesgos asociados a filtraciones de información sensible.

Regla de diseño

La contraseña original no puede recuperarse a partir del valor almacenado; el sistema únicamente valida credenciales comparando el hash generado en el proceso de autenticación.

El script completo de creación de la base de datos, que incluye la definición de tablas, claves y relaciones, se encuentra disponible en los anexos del presente documento como anexo para su consulta.

Diseño del sistema

El presente apartado describe la estructura estática del sistema mediante la utilización de diagramas de clases UML.

Se presentan dos niveles de modelado complementarios:

- Diagrama de clases del modelo conceptual
- Diagrama de clases del diseño técnico (Arquitectura MVC)

Esta separación responde a la necesidad de diferenciar claramente:

- La representación del dominio del problema.
- La arquitectura concreta adoptada para su implementación.

Dicha distinción permite mantener coherencia metodológica entre la etapa de análisis y la etapa de diseño, evitando mezclar conceptos de negocio con decisiones técnicas de implementación.

Diagrama de clases del modelo conceptual

El Diagrama de Clases del Modelo Conceptual representa las entidades principales del dominio del sistema Multiserv Stock, sus atributos relevantes y las relaciones estructurales entre ellas. Este diagrama tiene como objetivo modelar el problema desde una perspectiva independiente de la tecnología, sin incluir detalles propios de la infraestructura, persistencia o arquitectura de software.

Se centra exclusivamente en:

- Las entidades del dominio.
- Sus atributos significativos.
- Las asociaciones y multiplicidades.
- La navegabilidad entre clases.

No se incluyen en este nivel:

- Controladores.
- Repositorios (DAO).
- Clases de infraestructura.
- Tecnologías específicas (PDO, sesiones, JSON, etc.).

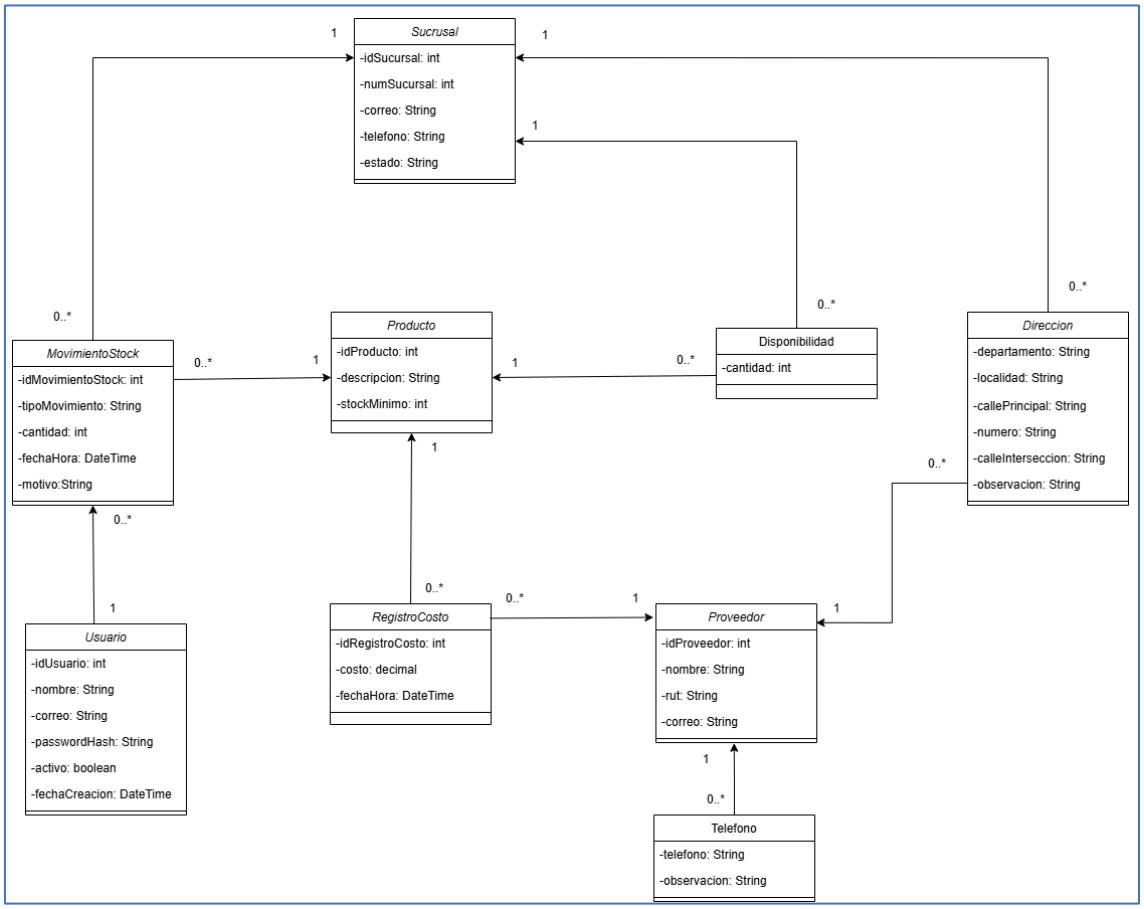


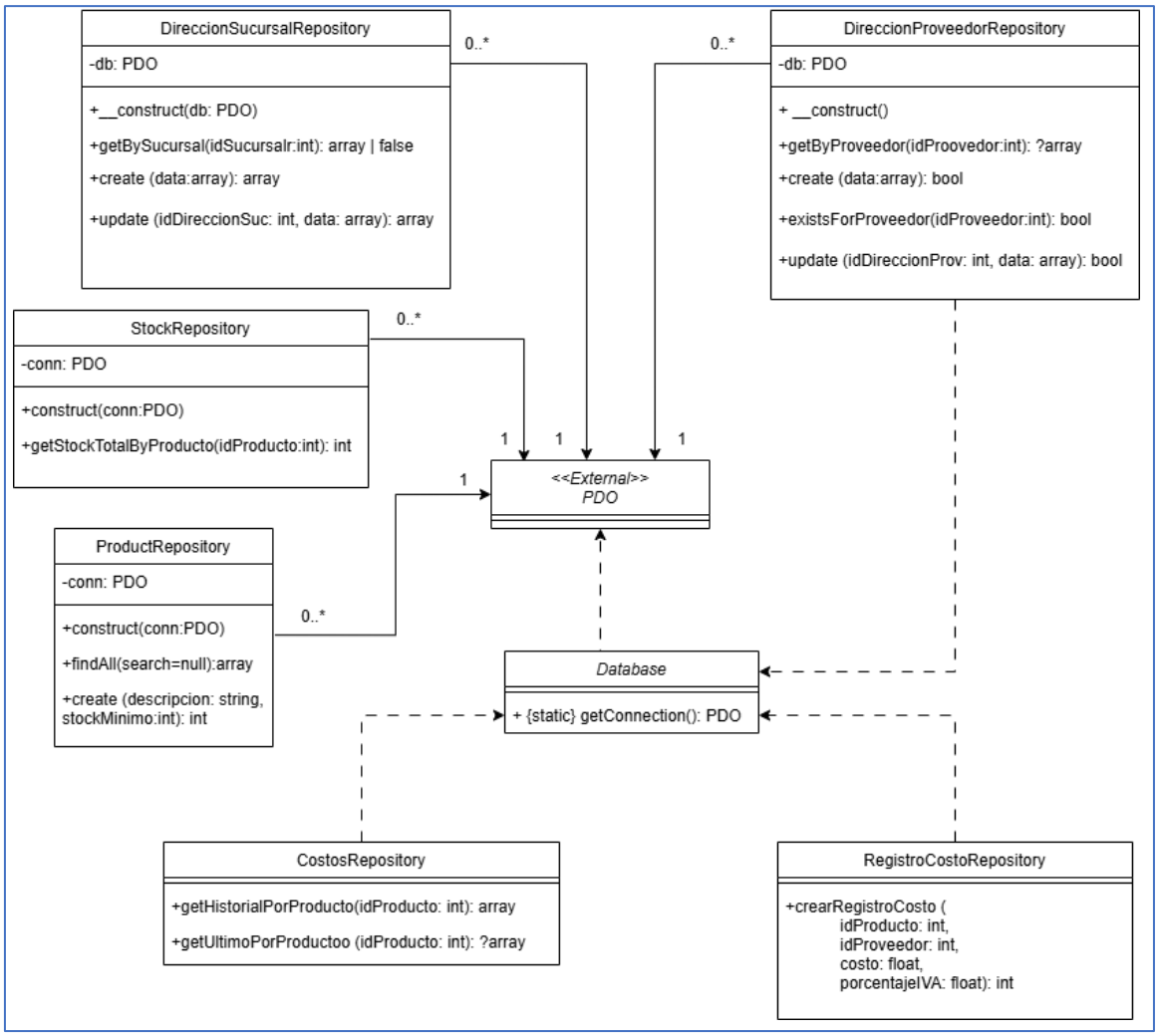
Diagrama de clases del diseño técnico (Arquitectura MVC)

El Diagrama de Clases del Diseño Técnico representa la materialización concreta del modelo conceptual dentro de la arquitectura implementada para el sistema Multiserv Stock.

A diferencia del modelo conceptual, que describe el dominio de manera independiente de la tecnología, este diagrama refleja explícitamente:

- Las clases reales implementadas en el código.
- La estructura de la capa de persistencia.
- La obtención y gestión de la conexión a base de datos.
- Las relaciones de asociación y dependencia entre componentes.
- Las cardinalidades técnicas derivadas de la implementación.

La arquitectura adoptada corresponde a un esquema MVC simplificado, donde los controladores HTTP (scripts) delegan la lógica de acceso a datos en repositorios (DAO), los cuales interactúan con la infraestructura de base de datos mediante la clase Database y la clase externa PDO.



Infraestructura: Database y PDO

La infraestructura de acceso a datos del sistema está compuesta por la clase Database y la clase externa PDO.

Clase Database

La clase Database tiene como responsabilidad principal proporcionar conexiones a la base de datos mediante el método estático:

{static} getConnection(): PDO

Esta clase no mantiene estado interno ni almacena una instancia persistente de PDO. Cada invocación del método getConnection() crea y retorna una nueva instancia de PDO.

Tipo de relación con PDO

La relación entre Database y PDO se modela como dependencia (línea punteada), ya que:

- Database utiliza PDO dentro del método.
- No posee un atributo de tipo PDO.
- No mantiene asociación estructural permanente.

Por este motivo, no se establecen cardinalidades en esta relación, dado que la dependencia representa uso temporal y no vínculo estructural persistente.

Clase externa PDO

La clase PDO (PHP Data Objects) pertenece a la biblioteca estándar de PHP y se modela en el diagrama como:

<<External>> PDO

Su inclusión en el diagrama permite representar explícitamente:

- Las asociaciones estructurales de los repositorios.
- El punto de acceso real a la base de datos.
- La separación entre componentes propios del sistema y componentes externos.

PDO constituye el elemento central de la infraestructura de persistencia, siendo utilizado por los repositorios para ejecutar consultas SQL y gestionar la comunicación con la base de datos.

Repositorios con inyección de dependencia por constructor

Dentro de la capa de persistencia se identifican repositorios que reciben la conexión a base de datos mediante inyección por constructor. Esta estrategia desacopla la obtención de la conexión del repositorio y permite reutilizar una misma instancia de PDO en distintos componentes.

Las clases que implementan este enfoque son:

- ProductRepository
- StockRepository
- DireccionSucursalRepository

Estructura común

Estas clases presentan:

- Un atributo estructural de tipo PDO (*private PDO $conn* o *private PDO $db*).
- Un constructor que recibe una instancia de PDO.
- Métodos públicos que ejecutan operaciones SQL utilizando dicha conexión.

Ejemplo estructural:

```
- conn: PDO
+ __construct(conn: PDO)
```

Tipo de relación con PDO

La relación se modela como asociación unidireccional sólida hacia PDO, dado que:

- El repositorio mantiene una referencia estructural permanente a la instancia de PDO.
- La conexión forma parte del estado interno del objeto.

Repositorios con obtención estática de conexión

Dentro de la capa de persistencia se identifican repositorios que no reciben la conexión mediante inyección por constructor, sino que la obtienen invocando directamente el método estático Database::getConnection().

Las clases que implementan esta estrategia son:

- CostosRepository
- RegistroCostoRepository

Características estructurales

Estas clases presentan las siguientes particularidades:

- No poseen un atributo estructural de tipo PDO.
- No reciben la conexión mediante constructor.
- Obtienen la conexión dentro de sus métodos invocando Database::getConnection().

Ejemplo conceptual de uso:

```
$pdo = Database::getConnection();
```

Tipo de relación con Database

La relación entre estos repositorios y Database se modela como dependencia (línea punteada), dado que:

- La clase utiliza el método de Database.
- No mantiene una referencia estructural permanente.
- No existe atributo de tipo Database.

Repositorio con estrategia mixta de obtención de conexión

La clase DireccionProveedorRepository presenta una estructura que combina dos tipos de relación dentro de la capa de persistencia.

Características estructurales

Esta clase:

- Posee un atributo estructural de tipo PDO:

  - db: PDO

- No recibe la conexión por parámetro.
- Obtiene la conexión dentro del constructor mediante la invocación:

  Database::getConnection()

Cardinalidades y coherencia arquitectónica

En el diagrama técnico, las cardinalidades se aplican únicamente en las asociaciones estructurales con la clase PDO, dado que en estos casos existe un atributo permanente que vincula las clases.

En dichas asociaciones se establece:

- Cada repositorio requiere exactamente una instancia de PDO.
- Una instancia de PDO puede ser compartida por múltiples repositorios.

Las relaciones de dependencia (líneas punteadas) no presentan cardinalidad, ya que representan uso temporal dentro de métodos y no vínculos estructurales persistentes.

La navegabilidad se determina por el principio de conocimiento: la flecha apunta hacia la clase utilizada. Así, los repositorios conocen a PDO o a Database, y Database conoce a PDO.

Arquitectura general del sistema

El sistema Multiserv Stock se implementa como una aplicación web basada en arquitectura cliente–servidor, donde un frontend desarrollado en HTML, CSS y JavaScript consume una API REST implementada en PHP, organizada internamente siguiendo principios del patrón MVC y utilizando MySQL como sistema gestor de base de datos.

El sistema adopta una arquitectura cliente–servidor basado en una API REST.

Internamente, la API se organiza siguiendo principios del patrón Modelo–Vista–Controlador (MVC) y una capa de repositorios para el acceso a datos.

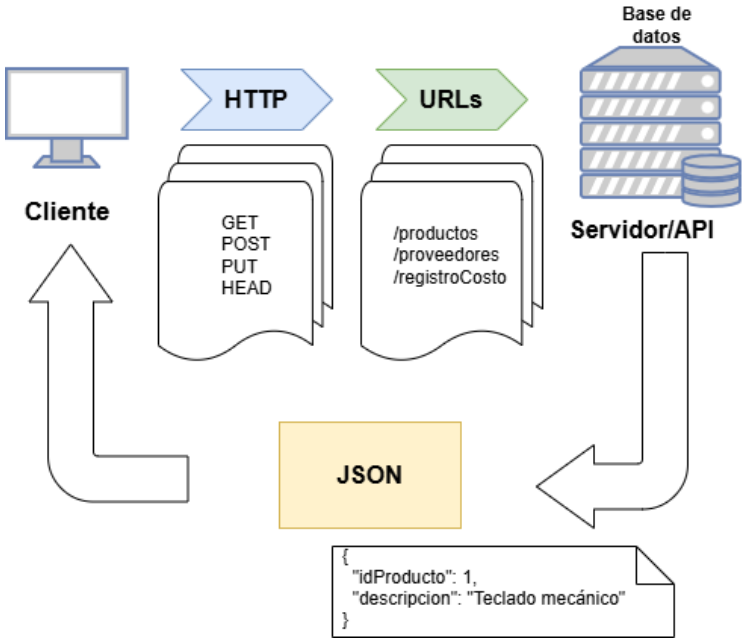
En esta arquitectura, la interfaz de usuario se implementa mediante páginas HTML y JavaScript, mientras que la lógica del sistema se centraliza en una API desarrollada en PHP, responsable de procesar las solicitudes del cliente y gestionar la comunicación con la base de datos.

Esta separación favorece el modularidad del sistema, la mantenibilidad del código y la posibilidad de extender o reutilizar componentes en futuras versiones.

A nivel conceptual, el sistema se organiza en capas desacopladas, donde la interacción con la base de datos se encuentra separada de la lógica de aplicación y de la interfaz de usuario.

Los detalles de implementación de esta arquitectura se desarrollan en el Anexo correspondiente.

Cada vista del sistema funciona de forma independiente, sin implementar navegación tipo SPA, priorizando simplicidad y desacoplamiento.



Arquitectura general de la API REST del sistema

Capa de presentación – Interfaz de usuario

El frontend del sistema Multiserv Stock tiene como objetivo proporcionar una interfaz simple y funcional que permita interactuar con la API REST desarrollada, facilitando la carga y consulta de información operativa.

Alcance del frontend

La interfaz implementada contempla funcionalidades de consulta y carga de datos, incorporando un mecanismo inicial de autenticación.

Diseño visual

Se definió una paleta de colores neutra con un color de acento, priorizando la legibilidad y la coherencia visual en todas las pantallas del sistema.

Reglas operativas de la interfaz

En la capa de presentación, el sistema implementa un conjunto de reglas operativas destinadas a validar los datos ingresados por el usuario y garantizar la consistencia de la información antes de su procesamiento por las capas inferiores.

- El código del producto es autogenerado por el sistema y no editable.
- La descripción del producto es obligatoria.

Estas reglas operativas sirven de base para el flujo de interacción del usuario con el sistema, el cual se describe en la sección siguiente.

Flujo del usuario – Gestión de productos

Esta sección describe el recorrido que realiza el usuario dentro del sistema Multiserv Stock para la gestión de productos, detallando las acciones disponibles, validaciones aplicadas y respuestas del sistema en cada etapa del proceso. El objetivo es garantizar una experiencia de uso clara, consistente y alineada con las reglas de negocio y el modelo de datos definidos.

Acceso a la gestión de productos

El usuario accede a la funcionalidad de Gestión de productos desde el menú principal del sistema.

Al ingresar, el sistema muestra una vista general con el listado de productos registrados, permitiendo visualizar información básica como:

- Código interno del producto
- Descripción
- Stock total disponible

Desde esta vista, el usuario puede realizar acciones de alta, consulta, edición o búsqueda de productos.

En el listado general se muestra el stock total de cada producto.

La disponibilidad detallada por sucursal puede consultarse mediante el modal de disponibilidad, donde se visualiza la cantidad correspondiente a cada sucursal registrada en el sistema.

Alta de un producto

Al seleccionar la opción *Agregar producto*, el sistema presenta un formulario de registro con los siguientes campos:

- Código del producto (autogenerado por el sistema)
- Descripción

Durante la carga del formulario, el sistema aplica validaciones en tiempo real:

- La descripción es obligatoria.
- El proveedor es obligatorio.
- El costo debe ser mayor a cero.

El botón *Guardar* permanece deshabilitado mientras existan datos inválidos o campos obligatorios incompletos.

Una vez confirmada la operación, el sistema registra el producto y muestra un mensaje indicando que el alta fue realizada correctamente.

La carga de stock inicial por sucursal se realiza mediante una funcionalidad independiente, asociada a la gestión de disponibilidad.

Consulta y visualización de productos

El usuario puede consultar los productos registrados a través del listado general o mediante herramientas de búsqueda y filtrado.

Al seleccionar un producto, el sistema muestra su información detallada, así como la disponibilidad correspondiente en cada sucursal, permitiendo verificar el estado actual del stock.

Edición de un producto

Desde la vista de detalle o el listado general, el usuario puede acceder a la opción de edición del producto.

El sistema permite modificar únicamente los campos habilitados, aplicando las mismas validaciones definidas para el alta.

Los cambios solo pueden guardarse cuando los datos cumplen con todas las reglas establecidas.

Respuesta del sistema y control de errores

Ante errores de validación o datos inconsistentes, el sistema informa al usuario mediante mensajes claros y específicos, indicando el motivo por el cual la acción no puede completarse. No se permiten registros parciales ni inconsistentes dentro del sistema.

De esta forma, el flujo de gestión de productos asegura un control adecuado de la información, manteniendo la coherencia entre la capa de presentación, la lógica del sistema y la base de datos.

Validación funcional de la interfaz

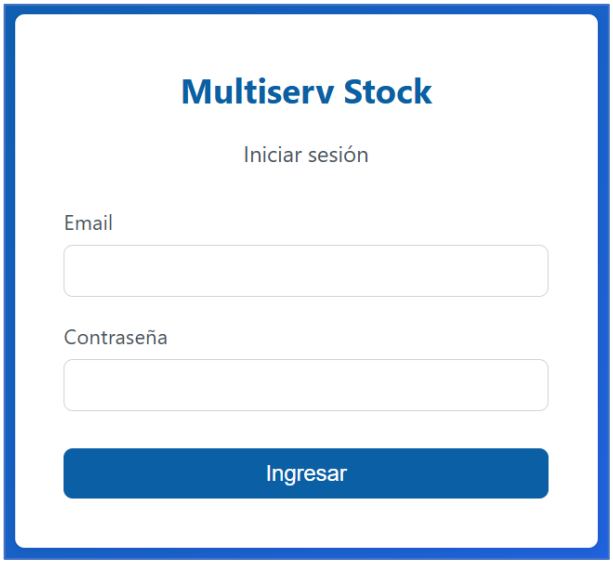
Con el objetivo de evidenciar el funcionamiento del sistema desarrollado, a continuación se presentan capturas de las principales funcionalidades implementadas en la interfaz de usuario. Estas imágenes ilustran las pantallas operativas del sistema y permiten visualizar el comportamiento general de la aplicación durante su uso.

La finalidad de esta sección es mostrar ejemplos representativos de interacción con el sistema, verificando la correcta implementación de las funcionalidades principales. No constituye un manual de uso detallado, ya que las instrucciones completas de operación se presentan posteriormente en el manual de usuario correspondiente.

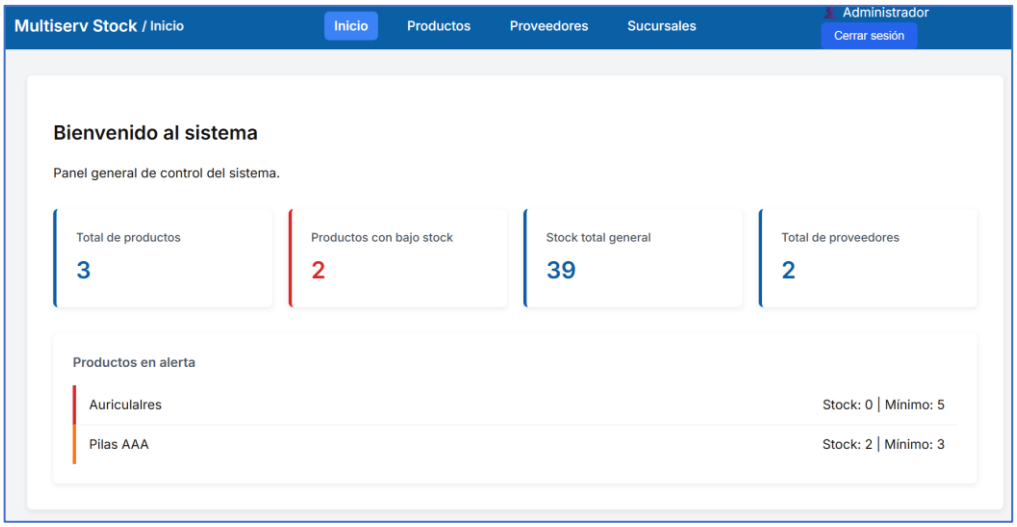
En las subsecciones siguientes se muestran las pantallas asociadas a:

1. inicio de sesión en el sistema
2. listado general de productos
3. alta de productos
4. visualización de disponibilidad de stock por sucursal

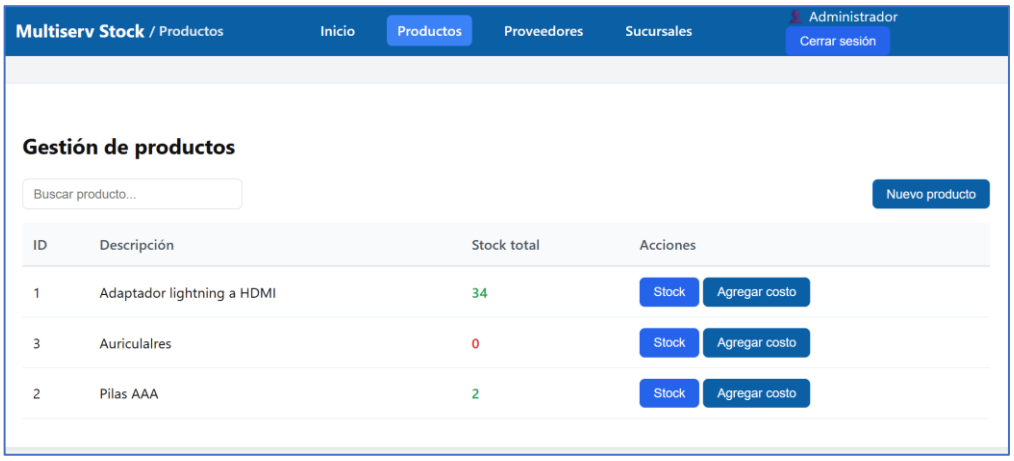
Inicio de sesión



Dashboard



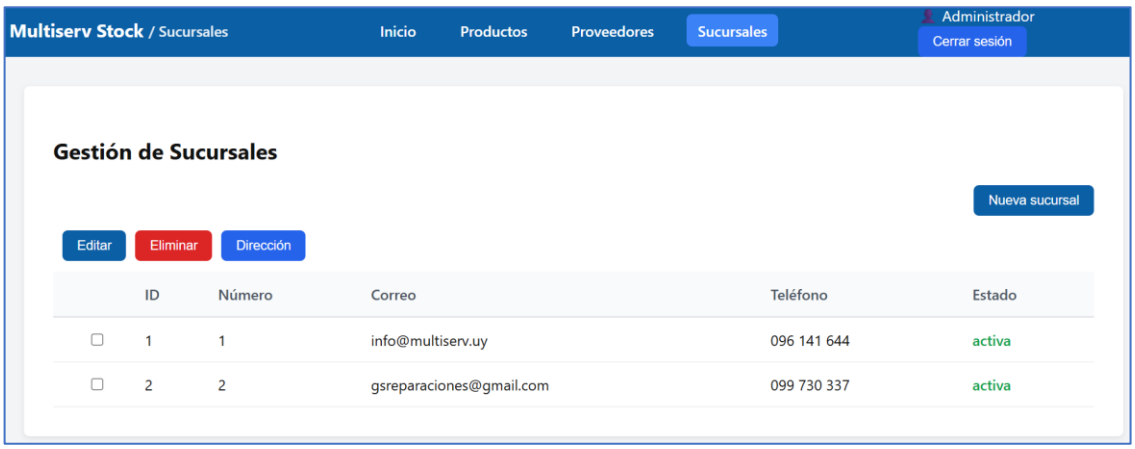
Lista de productos



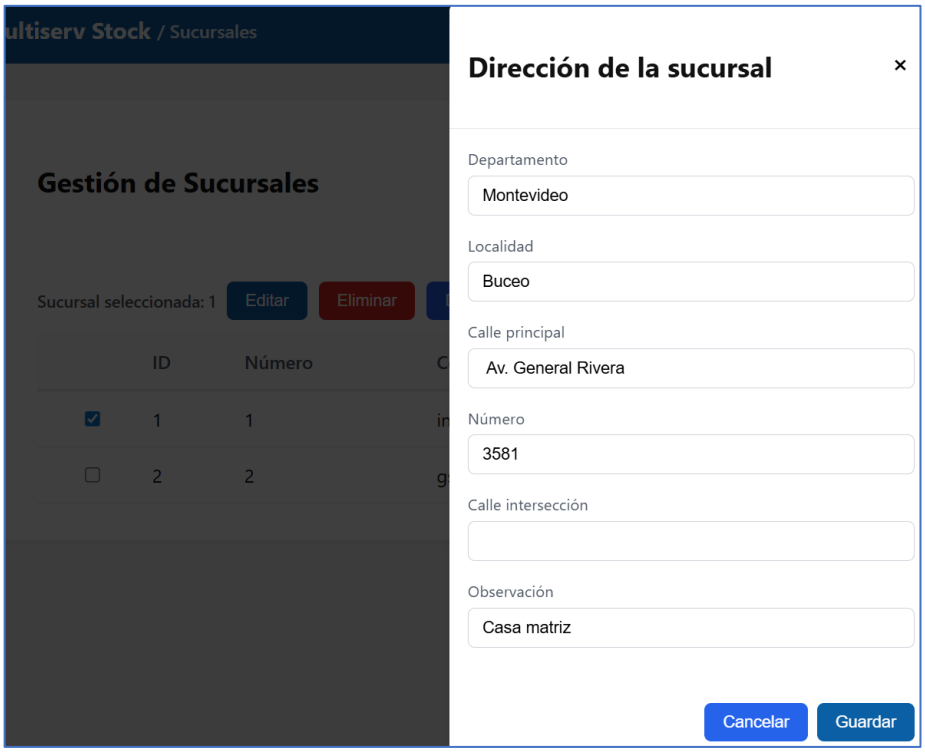
Nuevo producto

Visualización y edición de stock por sucursal

Visualización de sucursales



Visualización y edición de dirección por sucursal



Autenticación y seguridad

Inicialización del módulo de autenticación

Como parte de la implementación del módulo de seguridad, se creó la entidad usuario destinada a gestionar el acceso al sistema mediante autenticación segura.

Para la configuración inicial del sistema se procedió a:

Crear la tabla usuario con los siguientes campos:

- idUsuario (PK)
- nombre
- email (UNIQUE)
- passwordHash
- activo
- fechaCreacion

Generar un usuario administrador inicial mediante script temporal controlado.

La contraseña fue almacenada utilizando la función password_hash() de PHP, garantizando:

Uso de algoritmo seguro (bcrypt/Argon2 según versión PHP)

No almacenamiento en texto plano

Verificación posterior mediante password_verify()

El script de creación fue eliminado inmediatamente luego de su ejecución para evitar exposición no autorizada.

Consideraciones de seguridad

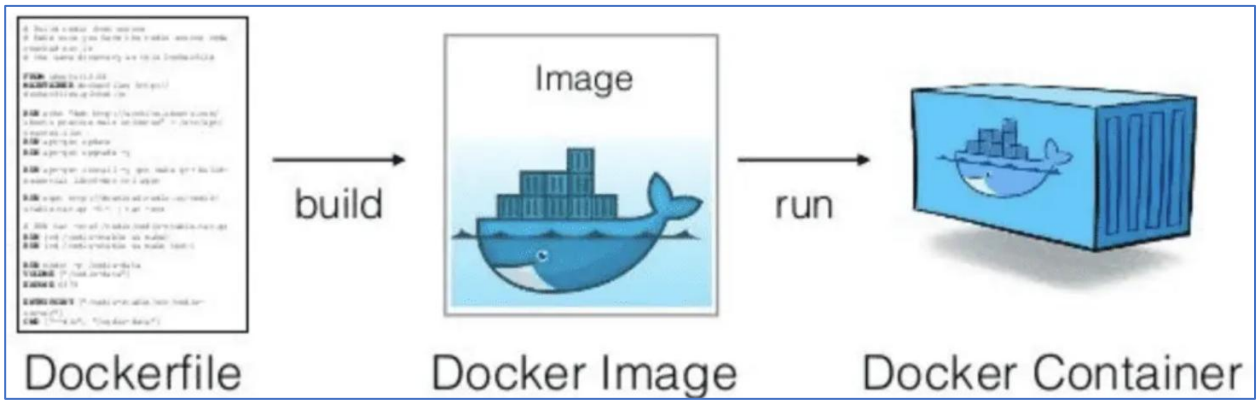
- Se implementa almacenamiento seguro de credenciales.
- No se exponen contraseñas en respuestas JSON.
- Se evita la persistencia de scripts de inicialización en entorno operativo.
- El campo email se define como UNIQUE para evitar duplicación de identidad.

Dockerización

Dockerizar un sistema consiste en encapsular la aplicación y su entorno de ejecución dentro de contenedores configurados mediante archivos declarativos, garantizando portabilidad, aislamiento y consistencia entre entornos de desarrollo y producción.

En el caso del sistema Multiserv Stock, la aplicación se apoya en el stack tecnológico comúnmente conocido como LAMP (Linux, Apache, MySQL y PHP). Este conjunto de tecnologías constituye una de las arquitecturas más utilizadas para el desarrollo de aplicaciones web.

A diferencia de una instalación tradicional donde todos los componentes se ejecutan dentro de un mismo servidor, en este proyecto dichos componentes se ejecutan en contenedores Docker independientes, lo que permite aislar la aplicación web y el sistema gestor de base de datos, facilitando su despliegue y mantenimiento.



El DocumentRoot es el directorio que el servidor web (Apache en este caso) expone públicamente cuando un usuario accede al dominio.

Es decir, define cuál es la carpeta que responde a:

https://dominio.com/

Todo lo que esté dentro del DocumentRoot puede ser accesible desde el navegador (según configuración), mientras que lo que esté fuera de él permanece protegido a nivel de servidor.

En el proceso de dockerización, configurar correctamente el DocumentRoot es fundamental para evitar exponer archivos internos del proyecto.

Justificación de la decisión adoptada

En el proceso de dockerización del sistema Multiserv Stock se establece el directorio *frontend-interno/* como DocumentRoot del contenedor web encargado de ejecutar Apache y PHP.

Esta configuración se define dentro del Dockerfile utilizado para construir el contenedor multiserv-web, mediante la variable de entorno APACHE_DOCUMENT_ROOT.

La elección de este directorio como punto de publicación del servidor web responde a las siguientes razones:

Separación clara entre Frontend y API

La carpeta frontend-interno/ contiene exclusivamente los archivos correspondientes a la interfaz del sistema (HTML, CSS y JavaScript), mientras que la API REST se encuentra implementada en el directorio api-stock/.

Esta organización preserva la distinción entre la capa de presentación y la capa de servicios del sistema, manteniendo coherencia con la arquitectura adoptada y facilitando la escalabilidad futura.

Seguridad mejorada

Al no exponer la raíz completa del proyecto como directorio público del servidor web, se evita el acceso directo a componentes internos de la aplicación.

Por ejemplo, directorios como:

- api-stock/src/
- frontend-interno/

contienen lógica de negocio, código fuente y componentes de la aplicación que no deben ser accesibles directamente desde el navegador.

En esta arquitectura, únicamente se expone el directorio correspondiente al frontend del sistema, mientras que el acceso a la API se realiza mediante un alias configurado en Apache que redirige las solicitudes hacia:

- api-stock/public

De esta forma se mantiene una separación clara entre la capa de presentación y la capa de servicios del sistema, reduciendo riesgos de exposición accidental de código interno.

Compatibilidad con entornos de producción

La configuración adoptada replica una práctica común en servidores web de producción, donde únicamente se publica el directorio público de la aplicación y el resto de los archivos permanecen fuera del alcance del servidor web.

Esto garantiza coherencia entre el entorno de desarrollo dockerizado y posibles entornos productivos basados en servidores Apache o Nginx.

Facilidad de migración a servidores VPS

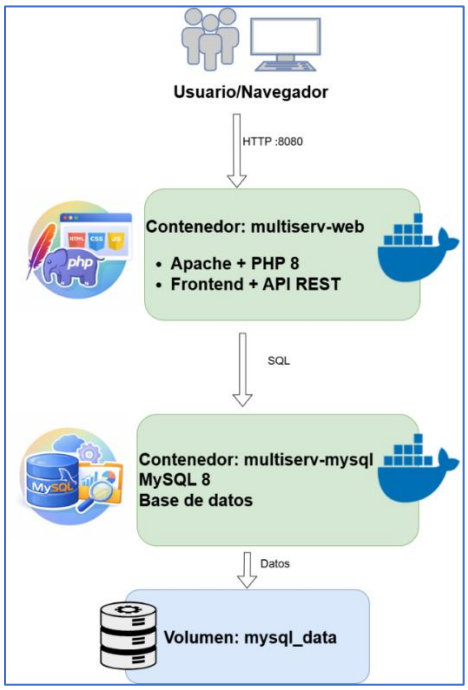
La estructura del proyecto permite migrar la aplicación a servidores tradicionales (VPS) sin necesidad de reorganizar el código ni modificar las rutas internas del sistema.

La separación entre directorio público y componentes internos del proyecto facilita la adaptación del sistema a distintos entornos de infraestructura.

Esta organización del proyecto facilita su ejecución dentro de contenedores Docker definidos mediante los archivos Dockerfile y docker-compose.yml, que permiten construir el contenedor web y orquestar su interacción con el contenedor de base de datos MySQL.

La siguiente figura muestra la arquitectura de contenedores utilizada para ejecutar el sistema mediante Docker. El contenedor multiserv-web ejecuta Apache y PHP y expone tanto la interfaz web como la API REST del sistema. Este contenedor se comunica mediante consultas SQL con el contenedor multiserv-mysql, encargado de ejecutar el sistema gestor de base de datos MySQL.

La persistencia de la información se garantiza mediante el volumen Docker mysql_data, que almacena los datos de la base de datos incluso si los contenedores se detienen o se reconstruyen.



Para facilitar la construcción y ejecución del sistema dockerizado, el proyecto incorpora distintos archivos de configuración que definen tanto la creación de las imágenes de contenedor como la orquestación de los servicios que componen el sistema.

Estos archivos se encuentran organizados dentro de la carpeta:

docker/

Entre los principales archivos utilizados se encuentran:

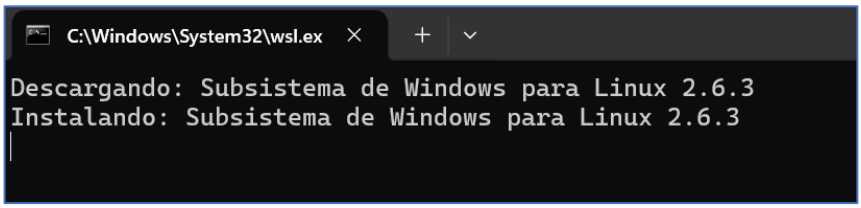
- Dockerfile, encargado de definir la imagen del contenedor que ejecuta el servidor web con PHP y Apache.
- docker-compose.yml, utilizado para orquestar los contenedores del sistema, incluyendo el servicio web y el servicio de base de datos.

Archivo	Ubicación	Función
Dockerfile	docker/Dockerfile	Define la construcción de la imagen del contenedor multiserv-web, instalando Apache, PHP y las extensiones necesarias para la conexión con MySQL. También configura el DocumentRoot y el alias para la API REST.
docker-compose.yml	docker/docker-compose.yml	Permite definir y orquestar los servicios que componen el sistema. En este proyecto se crean los contenedores multiserv-web y multiserv-mysql, además de configurar puertos, variables de entorno y volúmenes persistentes.
init.sql	docker/mysql/init.sql	Script SQL utilizado para inicializar automáticamente la estructura de la base de datos cuando el contenedor de MySQL se crea por primera vez.

Proceso de dockerización del sistema

Durante el desarrollo del proyecto, al utilizar un entorno Windows, fue necesario emplear WSL2 (Windows Subsystem for Linux) para ejecutar Docker sobre un entorno Linux integrado en el sistema.

Esto permitió trabajar con contenedores basados en Linux y garantizar compatibilidad con el stack tecnológico del sistema (Apache, PHP y MySQL).



Ubicados en la ruta del proyecto, dentro de la carpeta docker/, se ejecuta el siguiente comando para construir e iniciar los contenedores:

docker compose up –build

```
C:\xampp\htdocs\multiserv-stock\docker>docker compose up --build
time="2026-03-03T23:44:15-03:00" level=warning msg="C:\\xampp\\htdocs\\multiserv-stock\\docker\\doc
attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 10/14
 - Image mysql:8.0 [⠿⠿ ⠿⠿⠿⠿⠿⠿] 71.08MB / 247.3MB Pulling
```

```
ocation '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different
multiserv-mysql  | 2026-03-04T02:47:05.289643Z 0 [System] [MY-011323] [Server] X Plugin ready for c
ess: '::' port: 33060, socket: /var/run/mysqld/mysqlx.sock
multiserv-mysql  | 2026-03-04T02:47:05.289957Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: re
Version: '8.0.45'  socket: '/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community Server - GPL.

v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

Visualización de la base de datos

Para verificar el funcionamiento del contenedor de base de datos y acceder al motor MySQL que se ejecuta dentro del entorno Docker, se puede ingresar al contenedor mediante el siguiente comando:

docker exec -it multiserv-mysql mysql -u multiserv_user -p

Al ejecutar este comando, el sistema solicita la contraseña configurada para el usuario de la base de datos, correspondiente a:

multiserv_pass

Una vez autenticado, es posible consultar las tablas y verificar la estructura de la base de datos del sistema.

```
Símbolo del sistema - docker

Microsoft Windows [Versión 10.0.26200.7840]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\enalf>docker exec -it multiserv-mysql mysql -u multiserv_user -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```
mysql> USE multiserv;
Reading table information for completion of table a
You can turn off this feature to get a quicker star

Database changed
mysql> SHOW TABLES;
+---------------------+
| Tables_in_multiserv |
+---------------------+
| direccion_proveedor |
| direccion_sucursal  |
| disponibilidad      |
| movimiento_stock    |
| producto            |
| proveedor           |
| registro_costo      |
| sucursal            |
| telefono_proveedor  |
| usuario             |
+---------------------+
10 rows in set (0.01 sec)
```

Limitaciones de la versión

La presente versión del sistema ha sido desarrollada en función de un alcance acotado, priorizando la operativa básica de gestión de stock. En este contexto, se identifican las siguientes limitaciones:

- No se registra historial de movimientos de stock.
  Las actualizaciones de stock se realizan directamente sobre la disponibilidad actual, sin persistencia de las operaciones realizadas.
- No se incluye funcionalidad de edición de productos una vez creados.
  Las modificaciones requieren la creación de un nuevo registro.
- No se dispone de reportes ni exportación de datos.
  El sistema se limita a la visualización en pantalla de la información registrada.
- La gestión de usuarios no forma parte del alcance funcional del sistema.
  El acceso se realiza mediante un usuario previamente configurado, sin administración desde la interfaz.
- No se contemplan integraciones con sistemas externos ni automatización de procesos.
  El sistema funciona de manera independiente dentro de su entorno de ejecución.

Estas limitaciones responden a decisiones de diseño orientadas a garantizar la estabilidad, simplicidad y correcto funcionamiento del sistema en su versión inicial.
Se prevé como evolución futura la incorporación de funcionalidades que amplíen las capacidades del sistema, tales como el registro de movimientos de stock para trazabilidad, generación de reportes y administración de usuarios.

Proyecciones a versiones futuras

El sistema desarrollado en su versión actual (v1.0) cumple con los objetivos definidos en el alcance del proyecto, permitiendo la gestión de productos, proveedores, sucursales y control de stock de forma funcional y consistente.

No obstante, durante el proceso de análisis y desarrollo, se identificaron posibles líneas de evolución que podrían incorporarse en futuras versiones del sistema, en función de nuevas necesidades operativas o estratégicas de la empresa.

Entre las principales proyecciones se destacan:

- Incorporación de un módulo de gestión de usuarios con distintos niveles de acceso y roles definidos dentro del sistema.
- Implementación de un historial de movimientos de stock, que permita registrar entradas y salidas con trazabilidad completa (funcionalidad parcialmente contemplada a nivel de diseño).
- Integración del sistema con la página web institucional de la empresa, permitiendo visualizar disponibilidad de productos en tiempo real.
- Desarrollo de funcionalidades orientadas a proveedores, que les permitan consultar información vinculada al stock de sus productos.
- Generación de reportes automatizados y exportación de datos en formatos como PDF o Excel.
- Reincorporación o ampliación de funcionalidades de edición en la interfaz de usuario, que en la versión actual fueron simplificadas o limitadas intencionalmente para priorizar la estabilidad, claridad de uso y consistencia del sistema.
- Incorporación de tecnologías de identificación automática, como códigos de barras o códigos QR, que permitan agilizar el registro y consulta de productos dentro del sistema, facilitando tareas como control de stock, ingreso de mercadería y verificación de disponibilidad en sucursales mediante dispositivos móviles.
- Profundización en la adaptación del sistema a las particularidades operativas de la empresa, incorporando configuraciones y estructuras de datos específicas del negocio. En la versión actual, varios componentes fueron diseñados con un enfoque generalista, permitiendo su reutilización en distintos contextos.

En futuras versiones, se podría avanzar hacia una personalización más alineada con la realidad de Multiserv, por ejemplo:

- Definición de nombres específicos para las sucursales, en lugar de identificaciones genéricas (como “Sucursal 1” o “Sucursal 2”), permitiendo reflejar la estructura real de la empresa.
- Incorporación de atributos particulares para los productos, según su tipo o rubro, contemplando información relevante para la operativa de Multiserv.

Ajustes en la interfaz y en los registros de base de datos para adaptarse a terminología propia del negocio.

Esta evolución permitiría una mayor integración del sistema con los procesos reales de la empresa, mejorando su usabilidad y valor operativo.

Estas proyecciones no forman parte del alcance de la versión actual, pero constituyen posibles líneas de mejora que permitirían ampliar las capacidades del sistema en un contexto de evolución futura.

Bibliografía

Larman, C. (2005). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3.a ed.). Prentice Hall

Informática, T. en. (s. f.). *Programación Avanzada*. Fing.edu.uy. Recuperado 16 de septiembre de 2021, de https://www.fing.edu.uy/tecnoinf/mvd/cursos/progavan/material/teo/pavan-teorico06-analisis-modelado_dominio.pdf

Entidad-Relación, M. (s/f). Teórico 5 – Diseño conceptual (parte 1). Edu.uy. Recuperado el 16 de enero de 2026, de https://www.fing.edu.uy/tecnoinf/maldonado/cursos/bd1/materiales/teo/teorico05.pdf

Qué es un diagrama entidad-relación. (2026, enero 6). Lucidchart. Recuperado https://www.lucidchart.com/pages/es/que-es-un-diagrama-entidad-relacion

Pressman, R. S. (1995). *Ingenieria del software*. McGraw-Hill Interamericana. Recuperado el 16 de enero de 2026, de https://apps.utel.edu.mx/recursos/files/r161r/w25309w/Libroingenieria.PDF

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison Wesley. Recuperado https://ia802808.us.archive.org/8/items/fundamentosdesistemasdebasesdedatos/Fundamentos-de-Sistemas-de-Bases-de-Datos.pdf

Language reference. (s/f). Php.net. Recuperado el 21 de enero de 2026, de https://www.php.net/manual/en/langref.php

El paradigma de la Programación Orientada a Objetos en PHP y el patrón de arquitectura de Software MVC por Eugenia Bahit

Bahit, E. (2011). *POO y MVC en PHP: El paradigma de la programación orientada a objetos en PHP y el patrón de arquitectura MVC*. Compuzoft.

PHP: PDO - Manual. (s/f). Php.net. Recuperado el 10 de marzo de 2026, de https://www.php.net/manual/es/book.pdo.php

Medina, S. (2022, agosto 20). *Dockerizando una app con Node y Express*. Medium. https://medium.com/@enigmaticNerd/dockerizando-una-app-con-node-y-express-7dfeb5d8b0a4

Docs + quickstarts. (s/f). Render.com. Recuperado el 10 de marzo de 2026, de https://render.com/docs

Quick Start tutorial. (2026, marzo 10). Quick Start Tutorial | Railway Docs; Railway. https://docs.railway.com/quick-start

Quickstart. (2026, marzo 9). Docker Documentation; Docker Inc. https://docs.docker.com/compose/gettingstarted/

Tutorial Docker Compose. (s/f). Ander Fernández Jauregui - Machine Learning & Software Development. Recuperado el 10 de marzo de 2026, de https://anderfernandez.com/blog/tutorial-docker-compose/

Anexo A – Manual de usuario

El sistema Multiserv Stock es una aplicación web destinada a la gestión de stock de productos en distintas sucursales de la empresa.

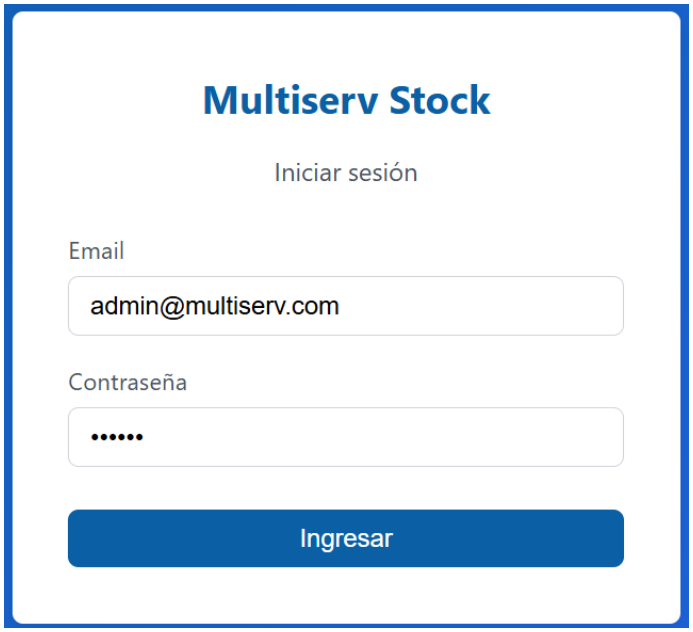
Permite registrar productos, proveedores y sucursales, así como consultar el stock disponible.

Este manual tiene como objetivo guiar al usuario en el uso de las principales funcionalidades del sistema.

Se recomienda el uso de navegadores actualizados como Google Chrome, Microsoft Edge o Mozilla Firefox.

Inicio de sesión

1. Ingresar a la página del sistema.
2. Introducir el correo electrónico.
3. Introducir la contraseña.
4. Presionar el botón Iniciar sesión.



Credenciales:

admin@multiserv.com

123456

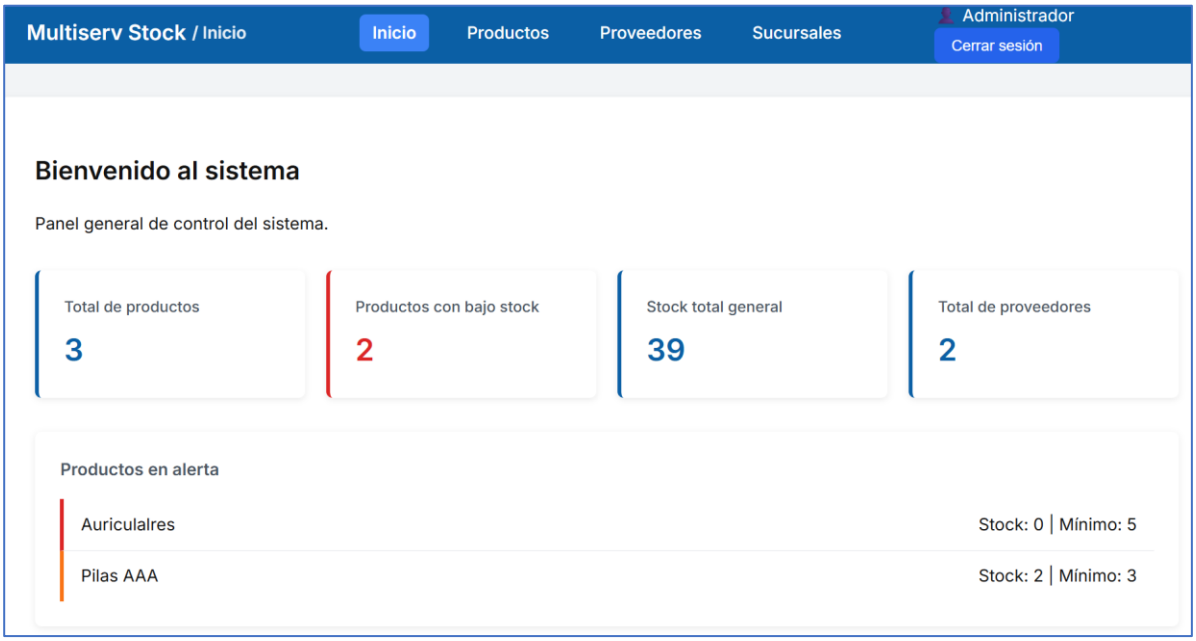
Panel principal

Una vez autenticado, el usuario accede al panel principal del sistema.

Desde allí puede navegar entre los distintos módulos disponibles mediante el menú superior.

Módulos principales:

- Inicio
- Productos
- Proveedores
- Sucursales



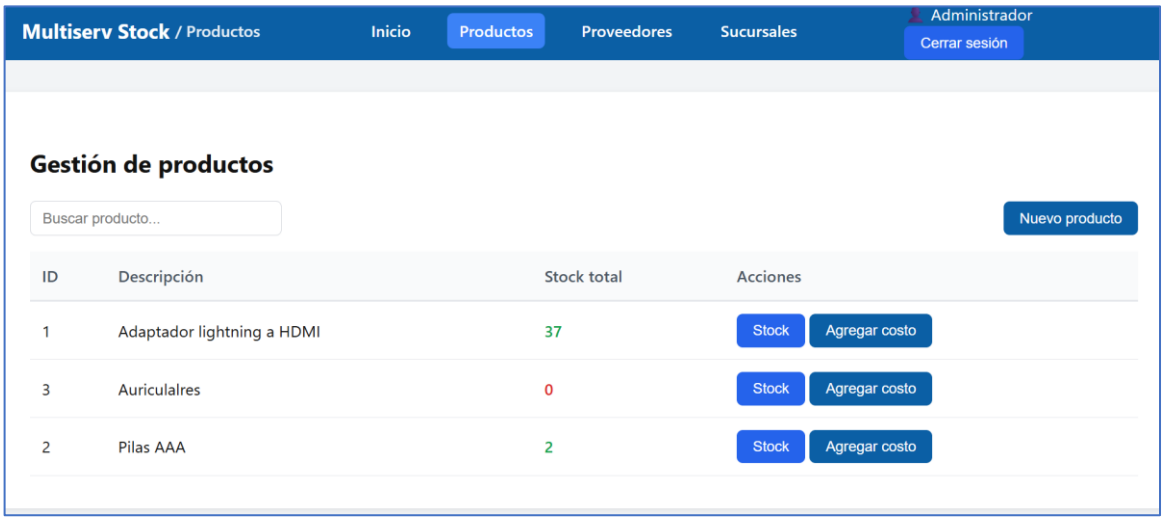
Gestión de productos

Visualizar productos

Al acceder al módulo de productos, se muestra un listado con todos los productos registrados.

El sistema muestra un listado con:

- Identificador del producto
- Descripción
- Stock total disponible

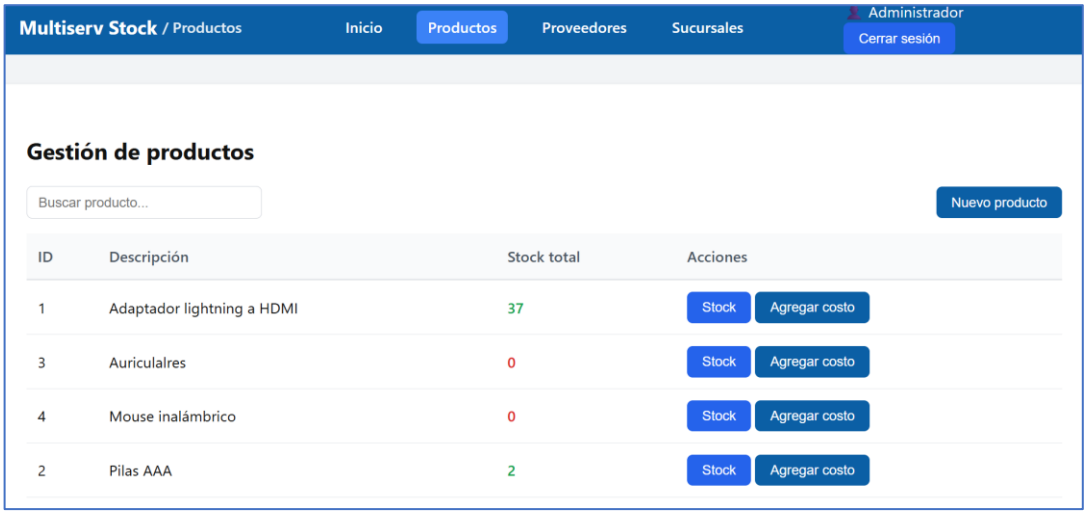


Agregar producto

Pasos:

- Acceder al módulo Productos.
- Presionar el botón Agregar producto.
- Completar los campos solicitados.
- Presionar Guardar.

El producto quedará registrado en el sistema.

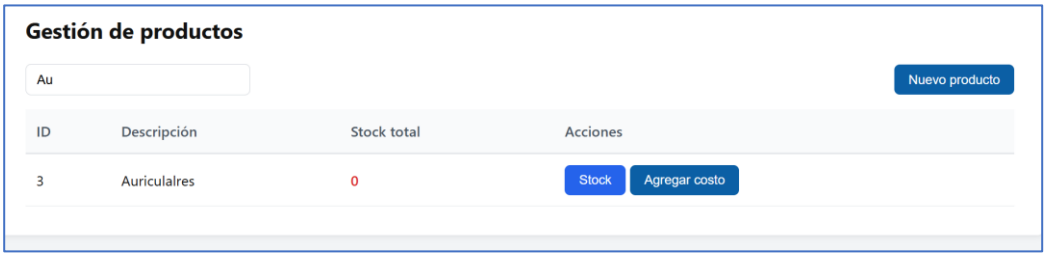


Buscar producto

El sistema permite filtrar productos mediante el campo de búsqueda.

Pasos:

- Ingresar texto en el campo Buscar producto.
- El sistema filtra automáticamente los resultados

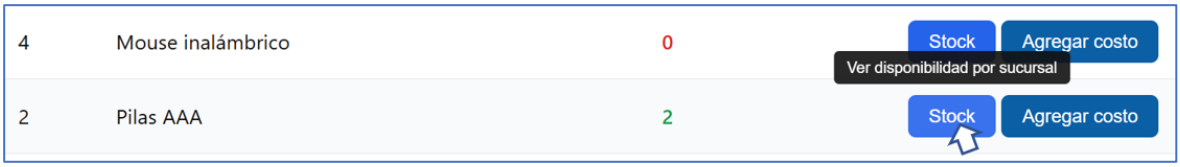


Gestión de stock

Cada producto dispone de la opción Stock, que permite gestionar la cantidad disponible.

Pasos:

- Presionar el botón Stock en el producto correspondiente.
- Registrar la operación correspondiente (ingreso o egreso).



Gestión de costos

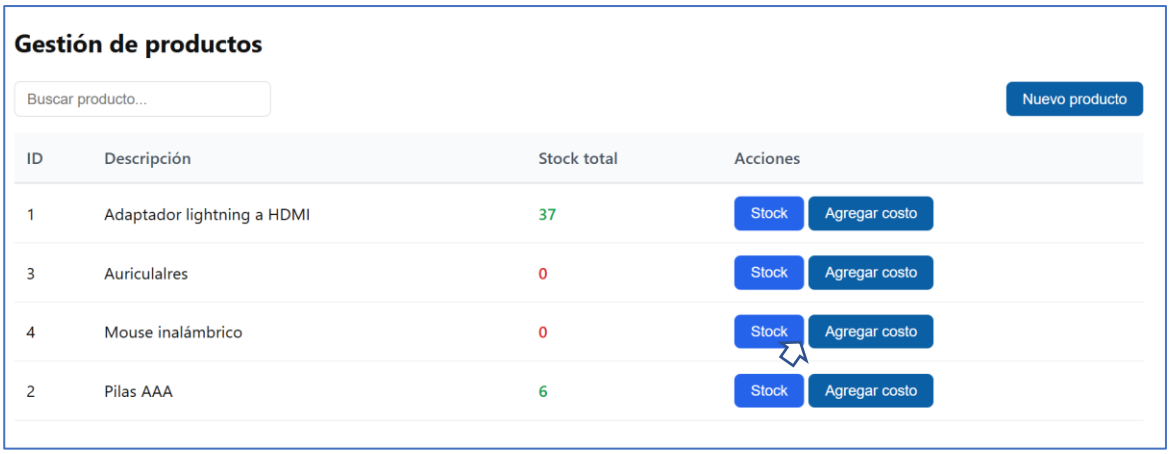
El sistema permite registrar costos asociados a los productos.

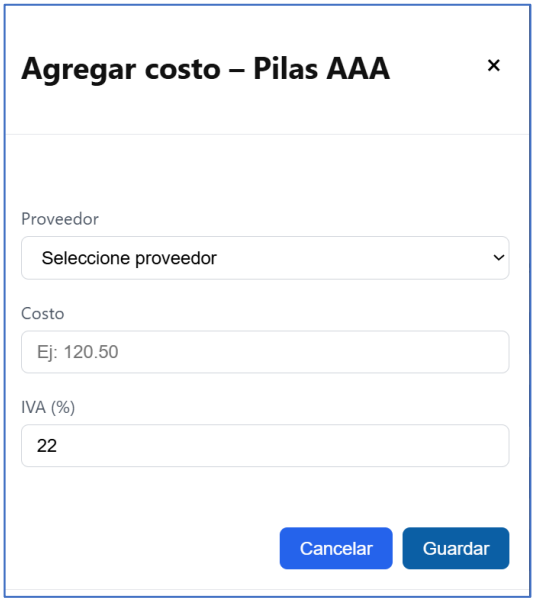
Cada producto dispone de la opción Agregar costo, que permite ingresar un nuevo valor.

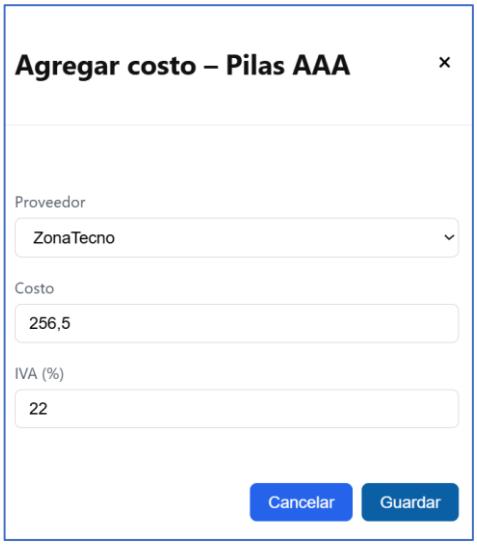
Pasos:

- Presionar el botón Agregar costo en el producto correspondiente.
- Seleccionar un proveedor.
- Ingresar el valor del costo.
- Ingresar el valor de IVA (%) asociado al producto.

  En caso de que el producto no tenga IVA, ingresar el valor 0.
- Confirmar la operación presionando el botón Guardar.







Gestión de proveedores

El módulo de proveedores permite administrar la información de los proveedores del sistema.

Visualizar proveedores

Al acceder al módulo, el sistema muestra un listado con los proveedores registrados.

La información visible incluye:

- Nombre del proveedor
- Datos de contacto
- Estado del proveedor

Buscar proveedor

El sistema permite filtrar proveedores mediante el campo de búsqueda.

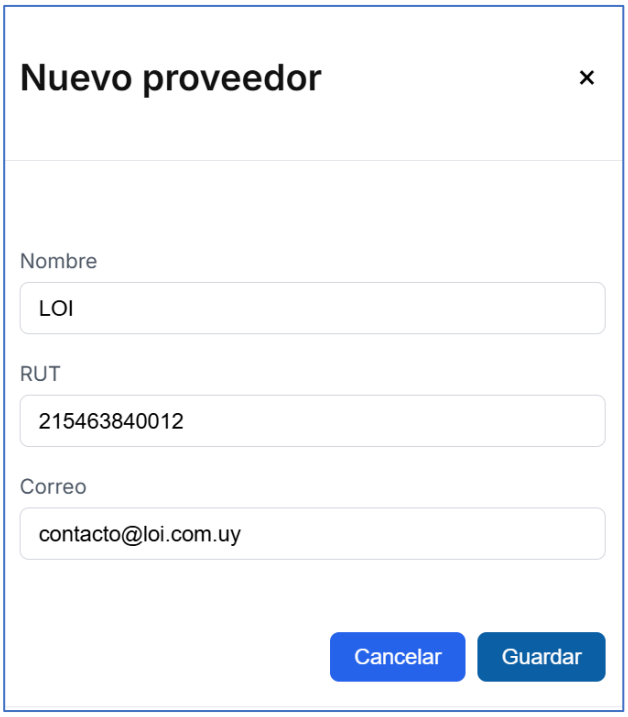
Pasos:

- Ingresar texto en el campo de búsqueda.
- El sistema filtra automáticamente los resultados.

Registrar proveedor

Pasos:

- Presionar el botón Nuevo proveedor.
- Completar los datos solicitados.
- Confirmar el registro.



Selección de proveedor

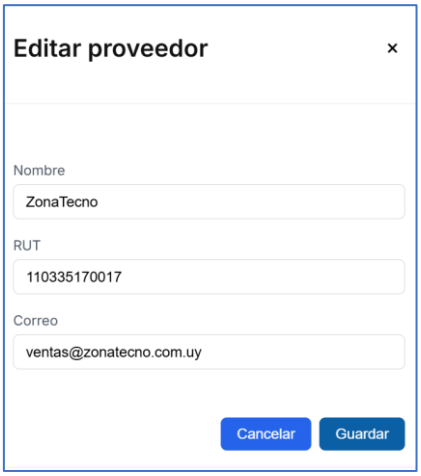
El sistema permite seleccionar un proveedor desde el listado.

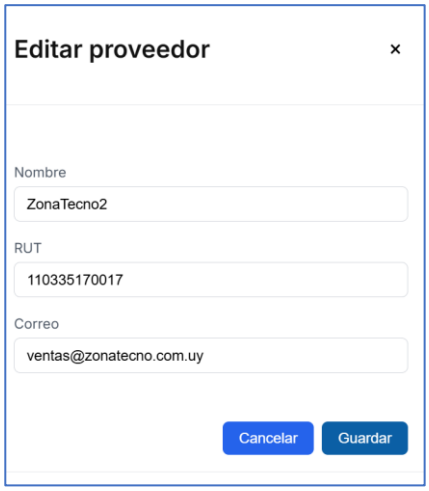
Al seleccionar un proveedor, se habilitan las acciones disponibles.

Editar proveedor

Pasos:

- Seleccionar un proveedor.
- Presionar el botón Editar.
- Modificar los datos necesarios.
- Guardar los cambios.





Activar / desactivar proveedor

El sistema permite cambiar el estado de un proveedor sin eliminarlo definitivamente.

Pasos:

- Seleccionar un proveedor.
- Presionar el botón Eliminar, según corresponda.

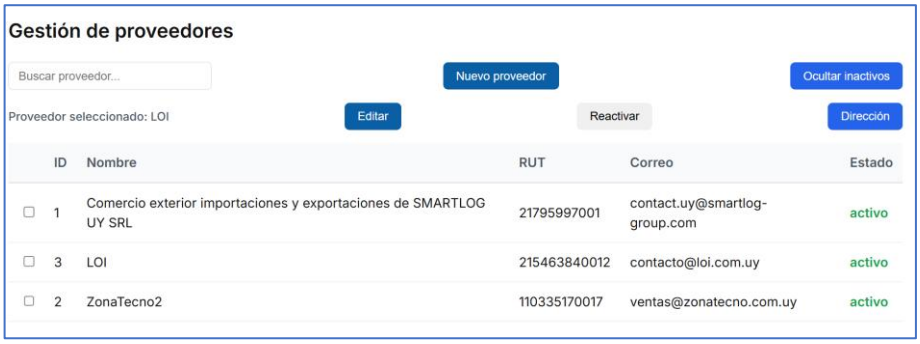
Esta acción modifica el estado del proveedor, manteniendo su información en el sistema.

Reactivar proveedor

Pasos:

- Presionar el botón Mostrar inactivos.
- Seleccionar el proveedor inactivo.
- Presionar el botón Reactivar

El proveedor vuelve a estado activo y se muestra nuevamente en el listado principal.



Gestión de dirección del proveedor

El sistema permite visualizar y registrar la dirección asociada a un proveedor.

Pasos:

- Seleccionar un proveedor.
- Presionar el botón Dirección.
- Completar los datos solicitados.
- Confirmar la operación presionando el botón Guardar.

En caso de existir una dirección previamente registrada, el sistema permite visualizarla y actualizar sus datos.

Gestión de sucursales

Visualizar sucursales

Al acceder al módulo, el sistema muestra un listado con las sucursales registradas.

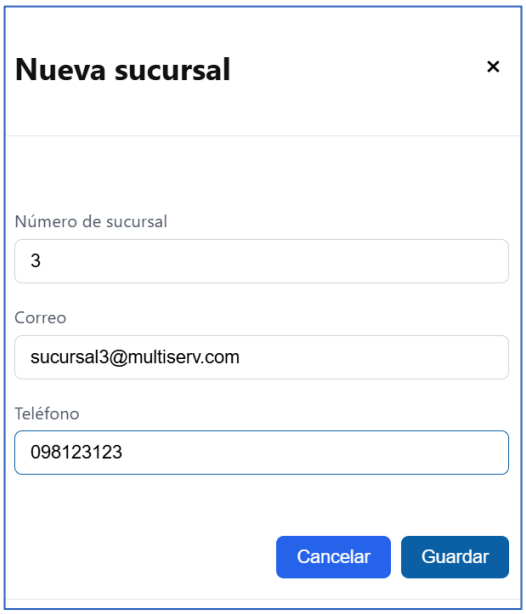
La información visible incluye:

- Número o identificador de sucursal
- Correo electrónico
- Teléfono
- Estado

Registrar sucursal

Pasos:

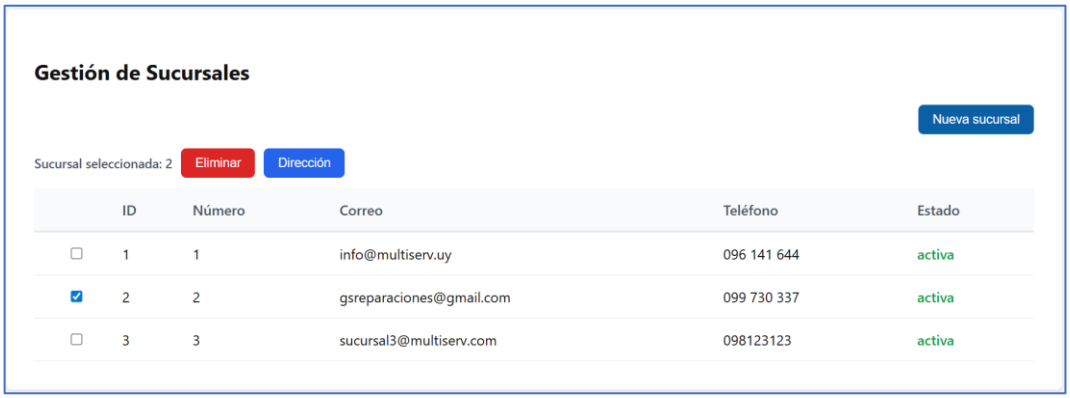
- Presionar el botón Nueva sucursal.
- Completar los datos solicitados.
- Confirmar el registro.



Selección de sucursal

El sistema permite seleccionar una sucursal desde el listado mediante la casilla correspondiente.

Al seleccionar una sucursal, se habilitan las acciones disponibles.

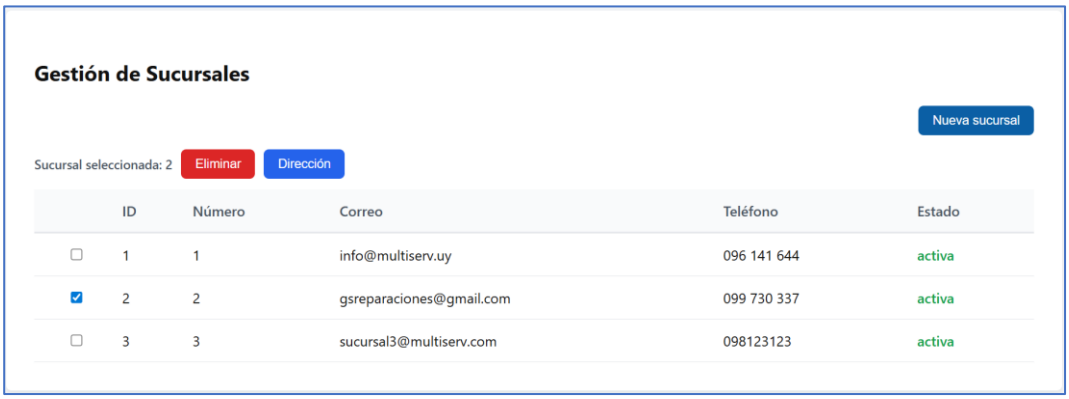


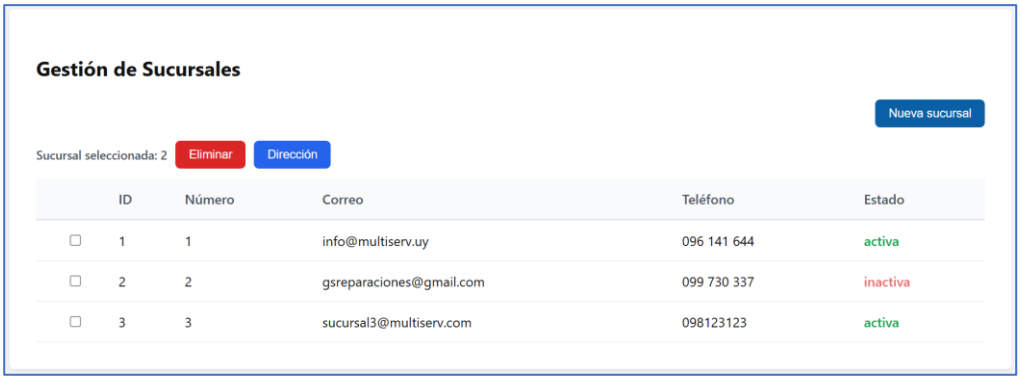
Cambio de estado de sucursal

El sistema permite modificar el estado de una sucursal sin eliminarla definitivamente.

Pasos:

- Seleccionar una sucursal.
- Presionar el botón Eliminar.



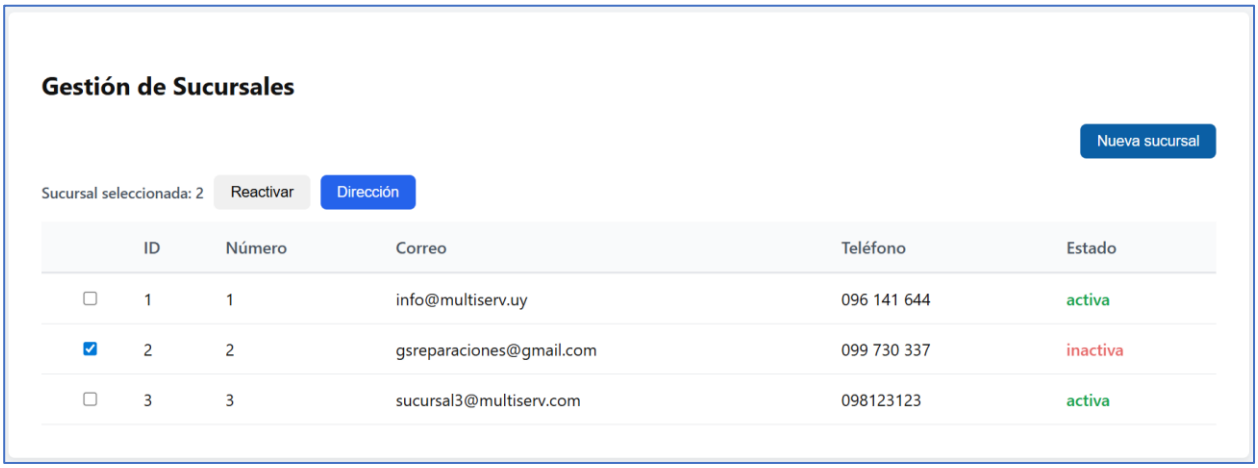


Reactivar sucursal

El sistema permite volver a activar una sucursal previamente desactivada.

Pasos:

- Seleccionar una sucursal inactiva.
- Presionar el botón Reactivar.

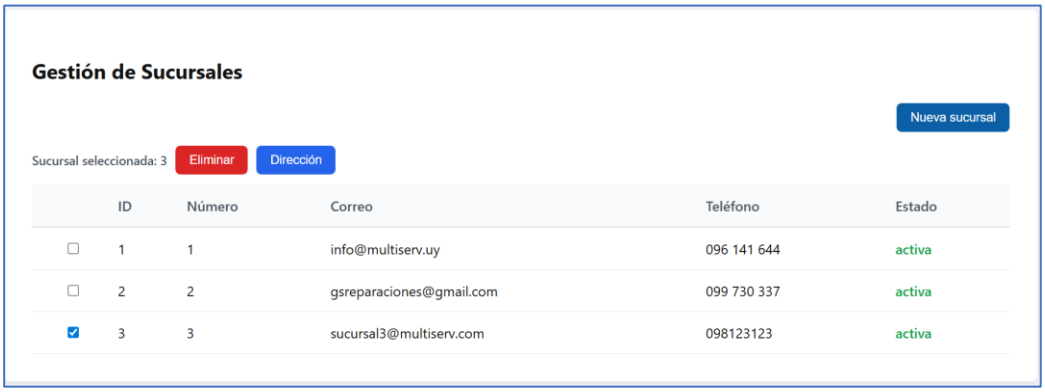


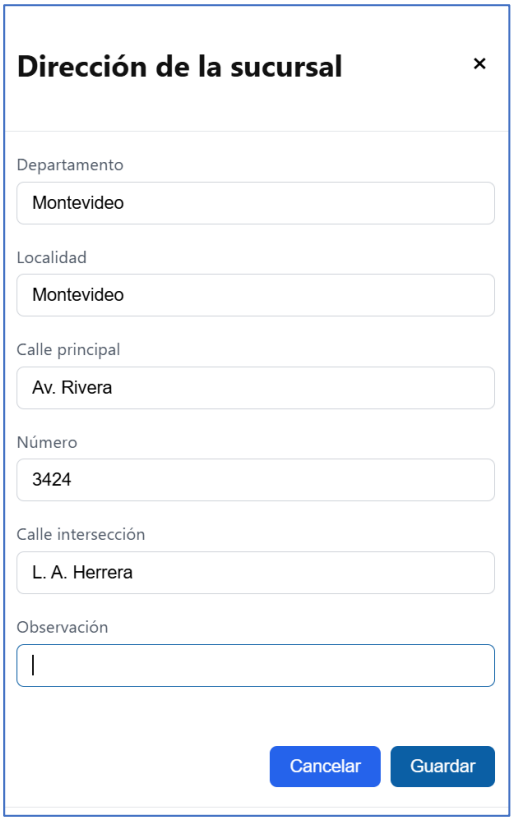
Gestión de dirección de la sucursal

El sistema permite visualizar y registrar la dirección asociada a una sucursal.

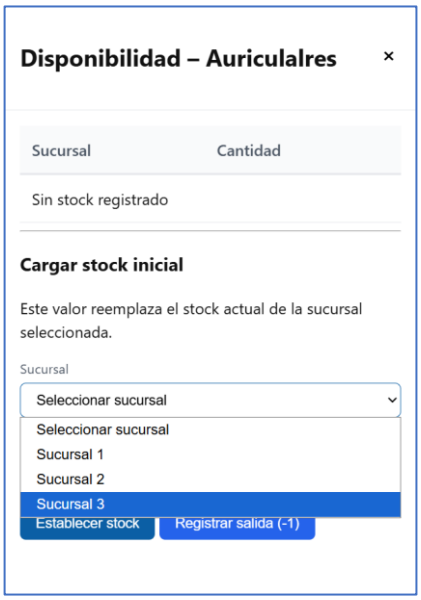
Pasos:

- Seleccionar una sucursal.
- Presionar el botón Dirección.
- Completar o modificar los datos de dirección.
- Confirmar la operación presionando el botón Guardar



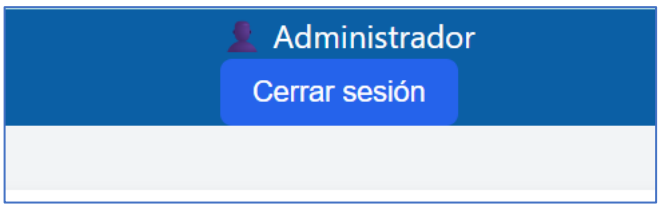


Se agrega la sucursal al menú de Stock de cada Producto



Cierre de sesión

El sistema permite finalizar la sesión del usuario de forma segura.



Anexo B - Modelo físico de la base de datos

El modelo físico de la base de datos se implementa sobre MySQL 8 a partir del modelo lógico definido previamente, respetando las claves primarias, foráneas y las restricciones de integridad referencial establecidas.

```
-- ============================================
-- MULTISERV STOCK - BASE DE DATOS COMPLETA
-- Versión 1.0
-- MySQL 8 - InnoDB
-- ============================================

-- Eliminación y recreación de la base

DROP DATABASE IF EXISTS multiserv_stock;

CREATE DATABASE multiserv_stock
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;

USE multiserv_stock;

-- ============================================
-- TABLA USUARIO
-- ============================================

CREATE TABLE usuario (
  idUsuario INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(120) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  passwordHash VARCHAR(255) NOT NULL,
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  fechaCreacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB;

-- ============================================
-- TABLA PRODUCTO
-- ============================================

CREATE TABLE producto (
  idProducto INT AUTO_INCREMENT PRIMARY KEY,
  descripcion VARCHAR(255) NOT NULL,
  stockMinimo INT NOT NULL DEFAULT 0,
  CHECK (stockMinimo >= 0)
) ENGINE=InnoDB;

-- ============================================
-- TABLA PROVEEDOR
-- ============================================

CREATE TABLE proveedor (
  idProveedor INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(120) NOT NULL,
  rut VARCHAR(20) NOT NULL,
  correo VARCHAR(255) NOT NULL,
  UNIQUE (rut),
  UNIQUE (correo)
) ENGINE=InnoDB;

-- ============================================
-- TABLA SUCURSAL
-- ============================================

CREATE TABLE sucursal (
  idSucursal INT AUTO_INCREMENT PRIMARY KEY,
```

```
  numSucursal INT NOT NULL,
  correo VARCHAR(255) NOT NULL,
  telefono VARCHAR(30) NOT NULL,
  estado ENUM('activa','inactiva') NOT NULL DEFAULT 'activa',
  UNIQUE (numSucursal),
  UNIQUE (correo)
) ENGINE=InnoDB;

-- ============================================
-- DIRECCION PROVEEDOR
-- ============================================

CREATE TABLE direccion_proveedor (
  idDireccionProv INT AUTO_INCREMENT PRIMARY KEY,
  idProveedor INT NOT NULL,
  departamento VARCHAR(100) NOT NULL,
  localidad VARCHAR(100) NOT NULL,
  callePrincipal VARCHAR(150) NOT NULL,
  numero VARCHAR(20),
  calleInterseccion VARCHAR(150),
  observacion VARCHAR(255),
  CONSTRAINT fk_dir_proveedor
    FOREIGN KEY (idProveedor)
    REFERENCES proveedor(idProveedor)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-- ============================================
-- DIRECCION SUCURSAL
-- ============================================

CREATE TABLE direccion_sucursal (
```

```
  idDireccionSuc INT AUTO_INCREMENT PRIMARY KEY,
  idSucursal INT NOT NULL,
  departamento VARCHAR(100) NOT NULL,
  localidad VARCHAR(100) NOT NULL,
  callePrincipal VARCHAR(150) NOT NULL,
  numero VARCHAR(20),
  calleInterseccion VARCHAR(150),
  observacion VARCHAR(255),
  CONSTRAINT fk_dir_sucursal
    FOREIGN KEY (idSucursal)
    REFERENCES sucursal(idSucursal)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-- ============================================
-- TELEFONOS PROVEEDOR
-- ============================================

CREATE TABLE telefono_proveedor (
  idTelefonoProveedor INT AUTO_INCREMENT PRIMARY KEY,
  idProveedor INT NOT NULL,
  telefono VARCHAR(30) NOT NULL,
  observacion VARCHAR(255),
  CONSTRAINT fk_tel_proveedor
    FOREIGN KEY (idProveedor)
    REFERENCES proveedor(idProveedor)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-- ============================================
-- REGISTRO COSTO
-- ============================================
```

```
CREATE TABLE registro_costo (
  idRegistroCosto INT AUTO_INCREMENT PRIMARY KEY,
  idProducto INT NOT NULL,
  idProveedor INT NOT NULL,
  costo DECIMAL(10,2) NOT NULL,
  fechaHora DATETIME NOT NULL,
  porcentajeIVA DECIMAL(5,2) NOT NULL,
  CHECK (costo > 0),
  CONSTRAINT fk_rc_producto
    FOREIGN KEY (idProducto)
    REFERENCES producto(idProducto),
  CONSTRAINT fk_rc_proveedor
    FOREIGN KEY (idProveedor)
    REFERENCES proveedor(idProveedor)
) ENGINE=InnoDB;

-- ============================================
-- DISPONIBILIDAD (estado actual)
-- ============================================

CREATE TABLE disponibilidad (
  idProducto INT NOT NULL,
  idSucursal INT NOT NULL,
  cantidad INT NOT NULL,
  PRIMARY KEY (idProducto, idSucursal),
  CHECK (cantidad >= 0),
  CONSTRAINT fk_disp_producto
    FOREIGN KEY (idProducto)
    REFERENCES producto(idProducto),
  CONSTRAINT fk_disp_sucursal
    FOREIGN KEY (idSucursal)
```

```
  REFERENCES sucursal(idSucursal)
) ENGINE=InnoDB;

-- ==========================================
-- MOVIMIENTO STOCK (histórico)
-- ==========================================

CREATE TABLE movimiento_stock (
 idMovimientoStock INT AUTO_INCREMENT PRIMARY KEY,
 idProducto INT NOT NULL,
 idSucursal INT NOT NULL,
 idUsuario INT NOT NULL,
 tipoMovimiento VARCHAR(50) NOT NULL,
 cantidad INT NOT NULL,
 fechaHora DATETIME NOT NULL,
 motivo VARCHAR(255),
 CONSTRAINT fk_ms_producto
  FOREIGN KEY (idProducto)
  REFERENCES producto(idProducto),
 CONSTRAINT fk_ms_sucursal
  FOREIGN KEY (idSucursal)
  REFERENCES sucursal(idSucursal),
 CONSTRAINT fk_ms_usuario
  FOREIGN KEY (idUsuario)
  REFERENCES usuario(idUsuario)
) ENGINE=InnoDB;

-- ==========================================
-- FIN DEL SCRIPT
-- ==========================================
```

Observaciones sobre las restricciones del modelo físico

En el modelo físico se incorporan restricciones propias del gestor de base de datos, orientadas a garantizar la consistencia y validez de la información almacenada.

UNIQUE

La restricción UNIQUE impide la existencia de valores duplicados en determinados atributos.

Se aplica, por ejemplo, al RUT y al correo electrónico del proveedor, así como al correo electrónico del usuario, asegurando que dichos datos identificatorios no puedan repetirse dentro del sistema.

NOT NULL

La restricción NOT NULL garantiza que ciertos atributos obligatorios no puedan almacenarse sin valor.

Se utiliza en aquellos campos cuya información resulta esencial para la integridad del modelo, como identificadores, descripciones, claves foráneas y datos requeridos para la operación del sistema.

CHECK

La restricción CHECK se emplea para validar condiciones específicas sobre valores numéricos, tales como la no negatividad del stock o del porcentaje de IVA.

FOREIGN KEY

Las claves foráneas aseguran la integridad referencial entre tablas, impidiendo la existencia de registros que referencien entidades inexistentes.

Validación de la implementación física

La implementación física de la base de datos fue validada mediante la ejecución completa del script SQL definido para el entorno MySQL 8, verificando:

- La correcta creación de todas las tablas y sus dependencias.
- La aplicación efectiva de claves primarias y foráneas.
- La activación de restricciones de integridad (UNIQUE, NOT NULL y CHECK).
- El uso del motor InnoDB en todas las estructuras, garantizando integridad referencial y soporte para transacciones.
- La correcta asociación de cada MovimientoStock a un Usuario mediante clave foránea, asegurando trazabilidad operativa a nivel estructural.

Asimismo, se comprobó la coherencia entre el modelo lógico documentado y el esquema físico generado, asegurando consistencia estructural completa entre análisis, diseño e implementación.

Descripción de la implementación física

La siguiente representación corresponde a la implementación física del modelo de datos en el gestor MySQL 8.

A través de la herramienta phpMyAdmin se visualiza la estructura real de la base de datos *multiserv_stock*, incluyendo las tablas creadas, sus relaciones y el motor de almacenamiento utilizado.

Esta visualización permite verificar la correspondencia entre el modelo lógico definido previamente y su materialización en el entorno de base de datos, asegurando la correcta aplicación de claves primarias, claves foráneas y restricciones de integridad.

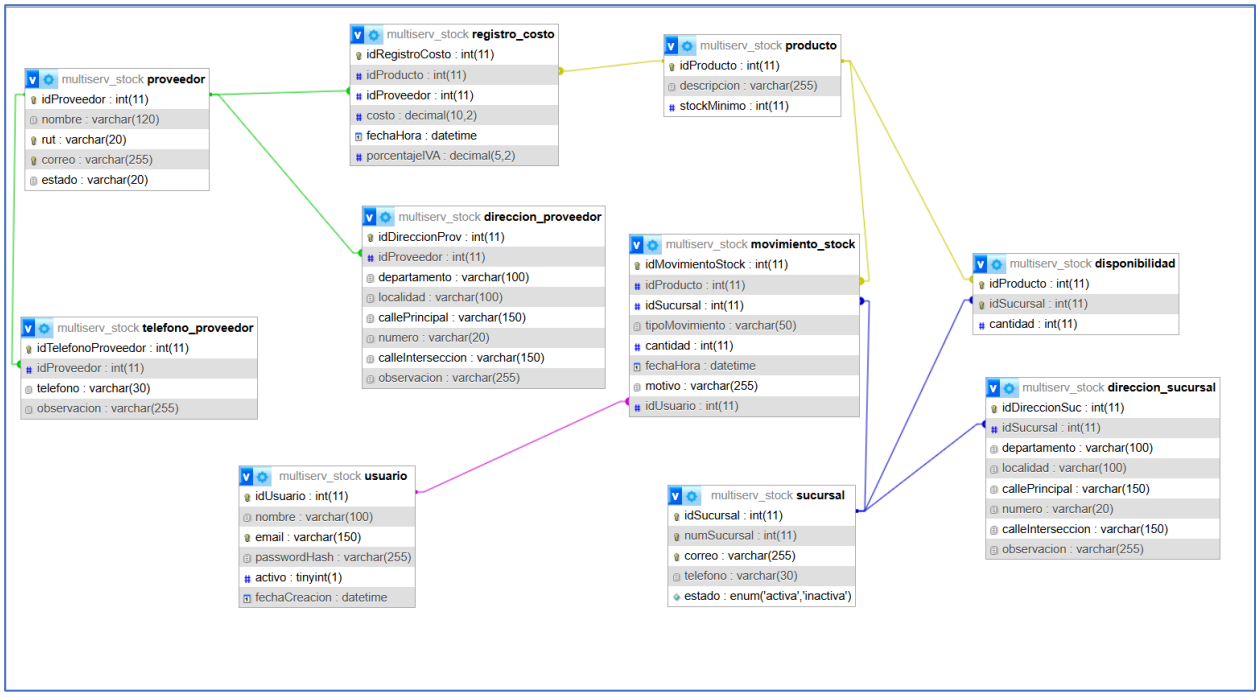
	Tabla	Acción						Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐	direccion_proveedor	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	8	InnoDB	utf8mb4_general_ci	32.0 KB	-
☐	direccion_sucursal	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	1	InnoDB	utf8mb4_general_ci	32.0 KB	-
☐	disponibilidad	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	9	InnoDB	utf8mb4_general_ci	32.0 KB	-
☐	movimiento_stock	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	1	InnoDB	utf8mb4_general_ci	64.0 KB	-
☐	producto	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	9	InnoDB	utf8mb4_general_ci	16.0 KB	-
☐	proveedor	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	16	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐	registro_costo	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	4	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐	sucursal	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	3	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐	telefono_proveedor	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	1	InnoDB	utf8mb4_general_ci	32.0 KB	-
☐	usuario	Examinar	Estructura	Buscar	Insertar	Vaciar	Eliminar	2	InnoDB	utf8mb4_general_ci	32.0 KB	-
	10 tablas	**Número de filas**						**54**	**InnoDB**	**utf8mb4_general_ci**	**384.0 KB**	**0 B**

Diagrama físico de la base de datos

El siguiente diagrama fue generado automáticamente a partir de la estructura de la base de datos mediante la herramienta phpMyAdmin.

Su objetivo es brindar una vista general de las tablas y sus relaciones físicas, facilitando la comprensión del esquema implementado.

Cabe aclarar que este diagrama representa la estructura física resultante y no reemplaza al Diagrama Entidad–Relación lógico presentado en secciones anteriores.


multiserv_stock proveedor
idProveedor : int(11)
nombre : varchar(120)
rut : varchar(20)
correo : varchar(255)
estado : varchar(20)
multiserv_stock registro_costo
idRegistroCosto : int(11)
idProducto : int(11)
idProveedor : int(11)
costo : decimal(10,2)
fechaHora : datetime
porcentajeIVA : decimal(5,2)
multiserv_stock producto
idProducto : int(11)
descripcion : varchar(255)
stockMinimo : int(11)
multiserv_stock direccion_proveedor
idDireccionProv : int(11)
idProveedor : int(11)
departamento : varchar(100)
localidad : varchar(100)
callePrincipal : varchar(150)
numero : varchar(20)
calleInterseccion : varchar(150)
observacion : varchar(255)
multiserv_stock movimiento_stock
idMovimientoStock : int(11)
idProducto : int(11)
idSucursal : int(11)
tipoMovimiento : varchar(50)
cantidad : int(11)
fechaHora : datetime
motivo : varchar(255)
idUsuario : int(11)
multiserv_stock disponibilidad
idProducto : int(11)
idSucursal : int(11)
cantidad : int(11)
multiserv_stock telefono_proveedor
idTelefonoProveedor : int(11)
idProveedor : int(11)
telefono : varchar(30)
observacion : varchar(255)
multiserv_stock direccion_sucursal
idDireccionSuc : int(11)
idSucursal : int(11)
departamento : varchar(100)
localidad : varchar(100)
callePrincipal : varchar(150)
numero : varchar(20)
calleInterseccion : varchar(150)
observacion : varchar(255)
multiserv_stock usuario
idUsuario : int(11)
nombre : varchar(100)
email : varchar(150)
passwordHash : varchar(255)
activo : tinyint(1)
fechaCreacion : datetime
multiserv_stock sucursal
idSucursal : int(11)
numSucursal : int(11)
correo : varchar(255)
telefono : varchar(30)
estado : enum('activa','inactiva')

Anexo C - Implementación de la API y capa de acceso a datos (DAO)

La arquitectura general del sistema Multiserv Stock fue presentada en la sección correspondiente del documento principal, donde se describió el modelo cliente–servidor basado en HTTP y el intercambio de información en formato JSON.

En el presente anexo se profundiza en la implementación concreta de dicha arquitectura, describiendo la organización de la API, la estructura de los endpoints y la capa de acceso a datos basada en el patrón DAO (Data Access Object).

El objetivo de esta sección es documentar cómo el modelo lógico definido previamente se materializa en código, manteniendo separación de responsabilidades y coherencia estructural.

La arquitectura general del sistema fue presentada en la sección “Arquitectura general del sistema”, donde se describió el modelo cliente–servidor basado en HTTP y el intercambio de datos en formato JSON.

En el presente anexo se profundiza exclusivamente en la implementación concreta de dicha arquitectura, detallando la organización de los endpoints y la capa de acceso a datos basada en el patrón DAO.

Organización de la API

La implementación se estructura en dos directorios principales:

/public

Contiene los endpoints accesibles mediante HTTP.

Cada archivo en este directorio representa un punto de entrada a la API y se encarga de:

Recibir la solicitud HTTP.

Validar datos de entrada.

Verificar autenticación cuando corresponde.

Delegar la operación a la capa DAO.

Devolver una respuesta en formato JSON.

/src

Contiene las clases de acceso a datos (DAO) y la gestión de conexión a la base de datos.

En esta carpeta se implementan:

Database (gestión de conexión mediante PDO).

ProductRepository

StockRepository

CostosRepository

RegistroCostoRepository

DireccionProveedorRepository

DireccionSucursalRepository

Otras clases auxiliares necesarias.

La separación entre /public y /src permite mantener desacoplada la lógica de persistencia del acceso externo.

Patrón DAO (Data Access Object)

El sistema implementa el patrón DAO con el objetivo de encapsular todas las operaciones de acceso a la base de datos dentro de clases específicas.

- Cada clase DAO:
- Recibe una instancia de conexión PDO.
- Ejecuta consultas preparadas.
- Devuelve resultados estructurados.
- No contiene lógica de presentación ni manejo de interfaz.

Esta decisión de diseño permite:

- Reducir el acoplamiento entre capas.
- Facilitar pruebas unitarias.
- Mantener coherencia con el modelo lógico.
- Permitir la evolución futura hacia una arquitectura MVC más formal.

Seguridad implementada en la API

El sistema incorpora un mecanismo inicial de autenticación de usuarios.

Las principales medidas implementadas son:

- Tabla usuario con credenciales almacenadas mediante password_hash.
- Verificación de credenciales mediante password_verify.
- Uso de consultas preparadas (PDO) para prevenir inyección SQL.
- Protección de endpoints mediante middleware de sesión.
- Asociación de cada movimiento de stock a un usuario autenticado.

El almacenamiento de la contraseña no se realiza en texto plano, sino mediante un hash seguro generado con el algoritmo definido por PASSWORD_DEFAULT de PHP.

Endpoints implementados

La API del sistema Multiserv Stock expone los siguientes endpoints REST.

Cada uno responde en formato JSON y utiliza consultas preparadas mediante PDO.

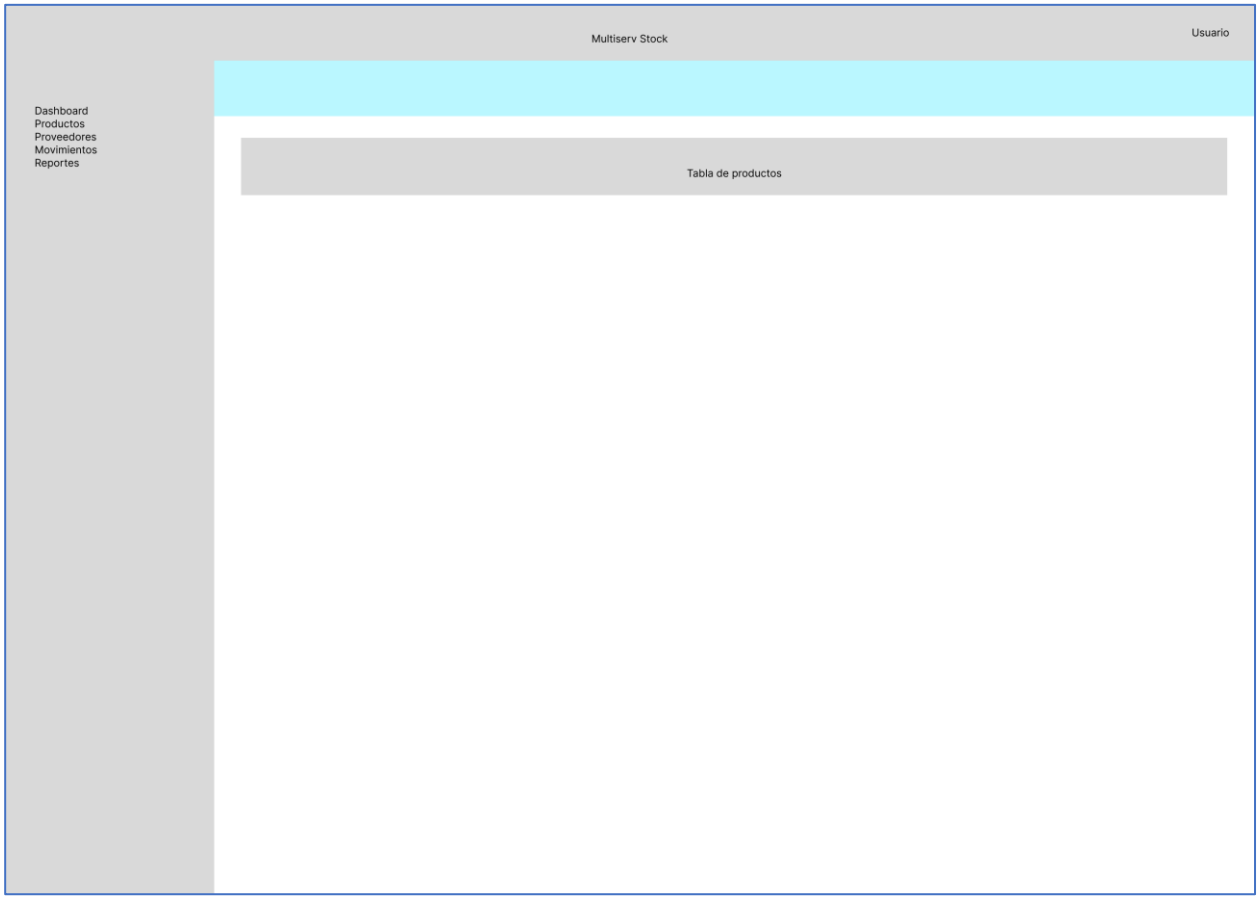
Método HTTP	Endpoint	Operación CRUD	Descripción funcional	Requiere autenticación
GET	/api-stock/public/productos.php	Read	Obtiene listado de productos con stock total calculado	Sí
POST	/api-stock/public/productos.php	Create	Registra un nuevo producto	Sí
GET	/api-stock/public/disponibilidad.php	Read	Consulta disponibilidad por producto y sucursal	Sí
POST	/api-stock/public/movimiento_stock.php	Create	Registra movimiento de stock y actualiza disponibilidad	Sí
GET	/api-stock/public/costos.php	Read	Obtiene historial de costos por producto	Sí
POST	/api-stock/public/login.php	Create (sesión)	Autentica usuario y crea sesión	No
POST	/api-stock/public/logout.php	Delete (sesión)	Cierra sesión activa	Sí

Anexo D - Diseño de Interfaz (UX / UI)

Wireframe

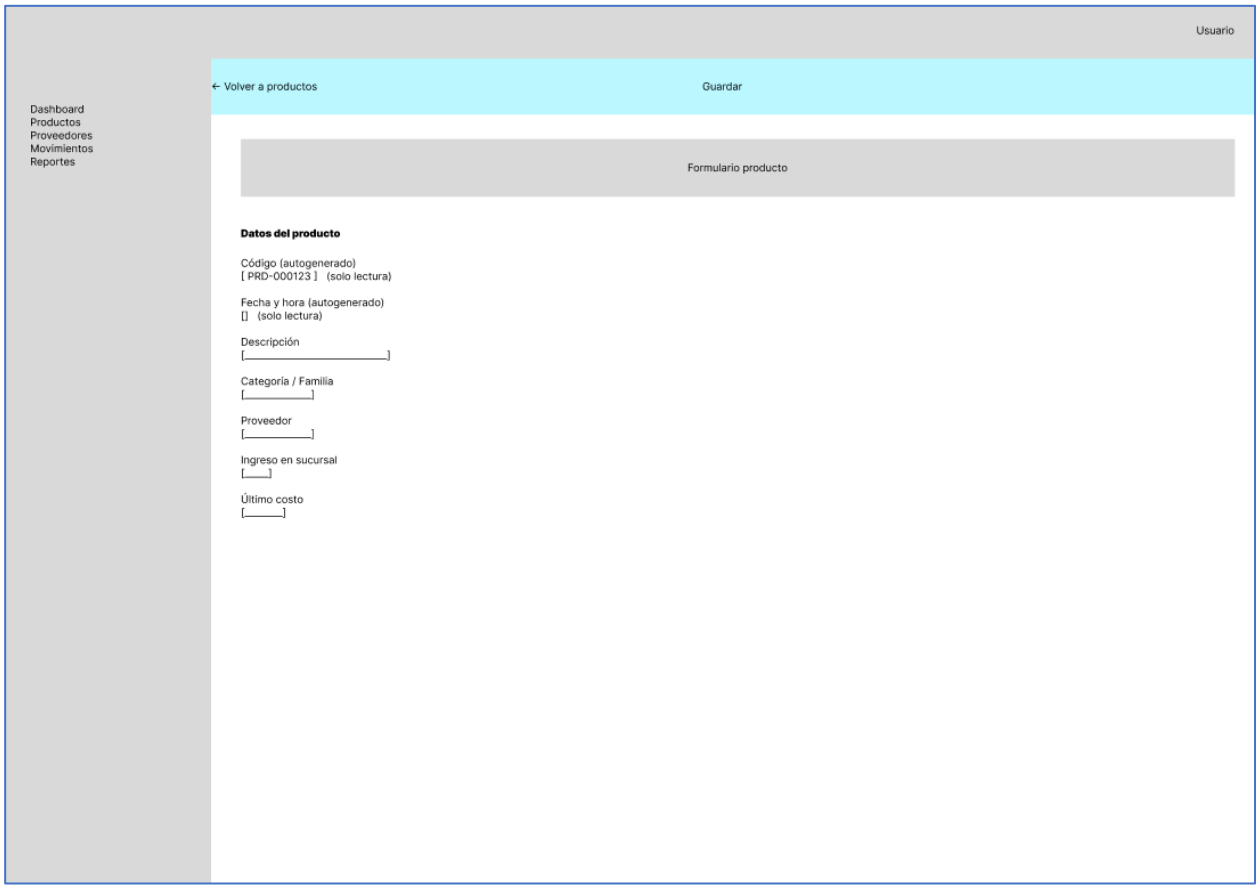
Los wireframes fueron realizados utilizando la herramienta Figma, y representan una primera versión conceptual de la interfaz del sistema. En esta etapa se priorizó la definición de la estructura, la jerarquía de la información y las acciones disponibles para el usuario, sin abordar aspectos visuales finales como colores o tipografías.

Pantalla Productos (listado)



Wireframe v1 de la pantalla de listado de productos.

Pantalla Producto – Alta / Edición



Wireframe v1 de la pantalla de alta/edición de Producto

Criterios de Interfaz de Usuario (UI)

La implementación de la paleta se realiza mediante variables CSS globales definidas en el archivo main.css, lo que permite mantener consistencia y facilitar modificaciones futuras.

<u>Colores principales</u>

Para la interfaz de usuario del sistema Multiserv Stock se adoptó la paleta institucional ya utilizada por la empresa Multiserv.Uy, con el objetivo de mantener coherencia visual e identidad corporativa entre los distintos sistemas de la organización.

No obstante, al tratarse de un sistema interno de gestión y no de una plataforma comercial, se optó por una aplicación sobria de dicha paleta, reduciendo el uso de colores de acento y elementos visuales promocionales, y priorizando la legibilidad, el contraste y la claridad de la información presentada.

Elemento	Color	
Header / navegación	#0B5FA5	
Fondo general	#F2F4F6	
Contenedor principal	#FFFFFF	
Texto principal	#111111	
Texto secundario	#4B5563	
Botón principal	#0B5FA5	
Botón secundario	#2563EB	
Acento (ocasional)	#FFD400	
Éxito	#16A34A	
Error	#DC2626	

Tipografía

Para la interfaz del sistema se seleccionó una tipografía sans serif de uso general, priorizando la legibilidad y la claridad de la información. Se optó por la fuente *Inter*, diseñada para interfaces digitales, por su buena lectura en distintos tamaños y su uso extendido en sistemas de gestión. En caso de no estar disponible, se prevé el uso de *Roboto* como alternativa compatible.

- Títulos: Inter / Semibold
- Texto general: Inter / Regular
- Botones: Inter / Medium

La fuente se encuentra disponible de forma gratuita a través de Google Fonts:

https://fonts.google.com/specimen/Inter

Anexo E - Despliegue

Pruebas de despliegue del sistema en servicios de nube (Render y Railway)

URL de acceso al sistema desplegado para pruebas.

El siguiente enlace corresponde a la instancia del sistema Multiserv Stock desplegada en la plataforma Render para fines de prueba y validación técnica.

https://multiserv-stock.onrender.com

Inicio de sesión:

admin@multiserv.com

123456

Las herramientas utilizadas durante el desarrollo y las pruebas del sistema, tales como Docker, la plataforma de despliegue Render y el servicio de base de datos Railway, fueron empleadas con el objetivo de verificar el funcionamiento del sistema en un entorno remoto distinto del entorno local de desarrollo.

Este despliegue permitió validar que la arquitectura implementada es capaz de ejecutarse correctamente fuera del entorno local, simulando condiciones más cercanas a un entorno productivo.

Las principales herramientas utilizadas fueron:

- Render
- Railway
- DBeaver

Uso de Render

https://render.com/

Render es una plataforma de servicios en la nube que permite desplegar aplicaciones web y APIs de forma sencilla a partir de un repositorio de código o mediante contenedores Docker.

En el proyecto Multiserv Stock, Render fue utilizado para desplegar el contenedor web del sistema, el cual ejecuta Apache y PHP, sirviendo tanto el frontend HTML del sistema como la API REST.

El despliegue en Render permitió:

- Verificar el funcionamiento del sistema en un servidor externo.
- Validar el acceso a la API desde el frontend mediante HTTP.
- Comprobar la correcta comunicación entre la aplicación y la base de datos remota.

El uso de Render facilitó la validación del sistema en un entorno de ejecución real sin necesidad de contar inicialmente con un servidor propio.

Uso de Railway

https://railway.com/

Railway es una plataforma de infraestructura en la nube que permite crear y administrar bases de datos y servicios backend de forma sencilla.

En este proyecto, Railway fue utilizado para alojar la base de datos MySQL del sistema, permitiendo que la API desplegada en Render pudiera conectarse a una base de datos remota.

La utilización de Railway permitió:

- Ejecutar MySQL en un entorno gestionado en la nube.
- Separar la capa de aplicación de la capa de datos.
- Validar la conexión entre servicios desplegados en distintas plataformas.

Esta separación de responsabilidades facilita la escalabilidad, mantenimiento y administración del sistema.

Uso de DBeaver

https://dbeaver.io/

DBeaver es una herramienta de administración de bases de datos que permite conectarse a múltiples motores de base de datos mediante interfaces gráficas.

En este proyecto, DBeaver fue utilizado para:

- Administrar la base de datos MySQL alojada en Railway.
- Verificar la estructura de las tablas.
- Ejecutar scripts SQL.
- Consultar los datos generados por el sistema durante las pruebas.

Es importante destacar que DBeaver no forma parte del flujo de ejecución del sistema, sino que se utiliza exclusivamente como herramienta de administración y verificación durante el desarrollo y las pruebas.

Representación arquitectónica

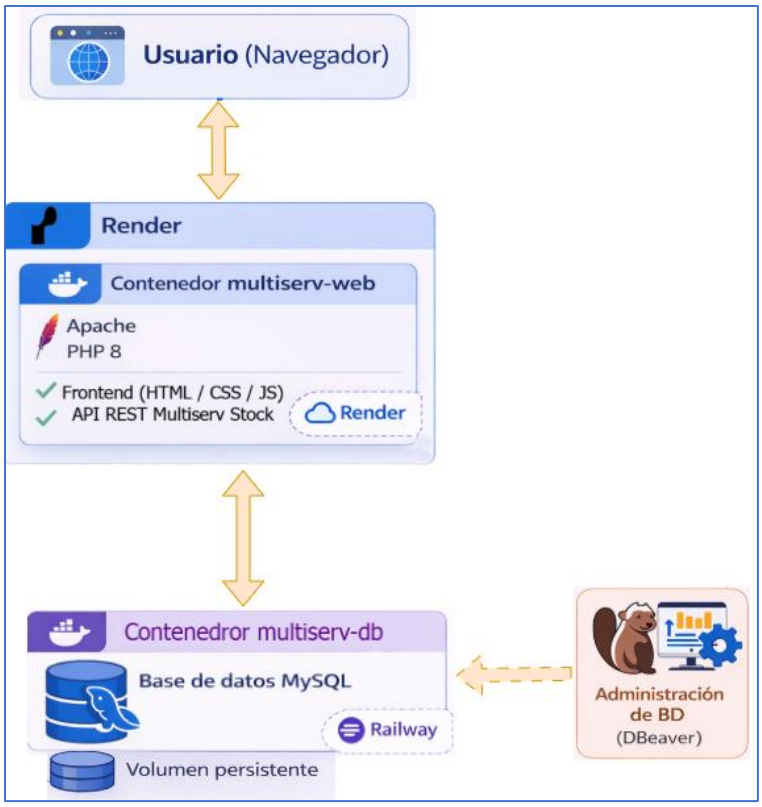
El siguiente diagrama representa la arquitectura de despliegue del sistema Multiserv Stock utilizando contenedores Docker.

El sistema se compone de dos contenedores principales:

- multiserv-web, encargado de ejecutar el servidor Apache junto con PHP 8, sirviendo tanto el frontend del sistema como la API REST que implementa la lógica de negocio.

- multiserv-db, que ejecuta MySQL y almacena los datos del sistema, utilizando un volumen persistente para garantizar la conservación de la información.

Para las pruebas de despliegue en entorno no local se utilizaron los servicios Render (para la aplicación) y Railway (para la base de datos), mientras que la administración de la base de datos se realiza mediante DBeaver.



Pruebas de concepto y ajustes realizados durante el despliegue

El despliegue del sistema Multiserv Stock en servicios de nube pública (Render y Railway) se realizó inicialmente como una prueba de concepto orientada al desarrollador, con el objetivo de verificar que la arquitectura del sistema pudiera ejecutarse fuera del entorno local de desarrollo.

Durante este proceso se detectaron y resolvieron diversos aspectos técnicos propios de la migración desde un entorno local (XAMPP) hacia una infraestructura en la nube.

Entre los principales ajustes realizados se destacan:

Adaptación del script de base de datos

El script original de creación de la base de datos fue modificado para eliminar instrucciones incompatibles con el entorno administrado de Railway, como la creación explícita de la base de datos (CREATE DATABASE). En estos servicios la base es creada automáticamente por la plataforma.

Ajustes en las variables de conexión

Fue necesario adaptar la configuración de conexión a MySQL para utilizar variables de entorno proporcionadas por Railway, reemplazando los valores de conexión locales utilizados durante el desarrollo.

Correcciones en rutas del sistema

Al desplegar la aplicación en Render se detectaron diferencias en las rutas relativas utilizadas por el frontend para acceder a la API. Estas rutas fueron ajustadas para garantizar el correcto acceso a los endpoints del sistema.

Verificación de autenticación y sesiones

Se realizaron pruebas para confirmar que el sistema de autenticación funcionara correctamente en un entorno remoto, verificando el manejo de sesiones y el acceso a endpoints protegidos mediante middleware de autenticación.

Validación del flujo completo del sistema

Una vez desplegada la aplicación, se realizaron pruebas funcionales sobre los principales módulos del sistema:

- gestión de productos
- gestión de proveedores
- gestión de sucursales
- registro de costos
- gestión de stock

Estas pruebas permitieron confirmar el correcto funcionamiento de la API y la persistencia de datos en la base de datos remota.

Adaptación del script SQL para Railway

El siguiente script corresponde a la estructura de tablas utilizada en la base de datos desplegada en Railway. El mismo fue exportado desde el gestor de base de datos DBeaver y posteriormente adaptado para eliminar instrucciones específicas del entorno local (como CREATE DATABASE), permitiendo su ejecución directa dentro del servicio MySQL gestionado por Railway.

```
-- ====================================================
-- MULTISERV STOCK
-- Script de inicialización de base de datos
-- Versión adaptada para Railway MySQL
```

```
-- ========================================================
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
-- ================================
-- TABLA USUARIO
-- ================================
CREATE TABLE usuario (
  idUsuario int NOT NULL AUTO_INCREMENT,
  nombre varchar(120) NOT NULL,
  email varchar(255) NOT NULL,
  passwordHash varchar(255) NOT NULL,
  activo tinyint(1) NOT NULL DEFAULT 1,
  fechaCreacion datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (idUsuario),
  UNIQUE KEY email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ================================
-- TABLA PROVEEDOR
-- ================================
CREATE TABLE proveedor (
  idProveedor int NOT NULL AUTO_INCREMENT,
  nombre varchar(120) NOT NULL,
  rut varchar(20) NOT NULL,
  correo varchar(255) NOT NULL,
  estado enum('activo','inactivo') NOT NULL DEFAULT 'activo',
  PRIMARY KEY (idProveedor),
  UNIQUE KEY rut (rut),
  UNIQUE KEY correo (correo)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ================================
```

```
-- TABLA PRODUCTO
-- ================================
CREATE TABLE producto (
  idProducto int NOT NULL AUTO_INCREMENT,
  descripcion varchar(255) NOT NULL,
  stockMinimo int NOT NULL DEFAULT 0,
  PRIMARY KEY (idProducto)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ================================
-- TABLA SUCURSAL
-- ================================
CREATE TABLE sucursal (
  idSucursal int NOT NULL AUTO_INCREMENT,
  numSucursal int NOT NULL,
  correo varchar(255) NOT NULL,
  telefono varchar(30) NOT NULL,
  estado enum('activa','inactiva') NOT NULL DEFAULT 'activa',
  PRIMARY KEY (idSucursal),
  UNIQUE KEY numSucursal (numSucursal),
  UNIQUE KEY correo (correo)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ================================
-- TABLA DIRECCION PROVEEDOR
-- ================================
CREATE TABLE direccion_proveedor (
  idDireccionProv int NOT NULL AUTO_INCREMENT,
  idProveedor int NOT NULL,
  departamento varchar(100) NOT NULL,
  localidad varchar(100) NOT NULL,
  callePrincipal varchar(150) NOT NULL,
```

```
  numero varchar(20),
  calleInterseccion varchar(150),
  observacion varchar(255),
  PRIMARY KEY (idDireccionProv),
  KEY fk_dir_proveedor (idProveedor),
  CONSTRAINT fk_dir_proveedor
    FOREIGN KEY (idProveedor)
    REFERENCES proveedor (idProveedor)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ================================
-- TABLA TELEFONO PROVEEDOR
-- ================================
CREATE TABLE telefono_proveedor (
  idTelefonoProveedor int NOT NULL AUTO_INCREMENT,
  idProveedor int NOT NULL,
  telefono varchar(30) NOT NULL,
  observacion varchar(255),
  PRIMARY KEY (idTelefonoProveedor),
  KEY fk_tel_proveedor (idProveedor),
  CONSTRAINT fk_tel_proveedor
    FOREIGN KEY (idProveedor)
    REFERENCES proveedor (idProveedor)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ================================
-- TABLA DIRECCION SUCURSAL
-- ================================
CREATE TABLE direccion_sucursal (
  idDireccionSuc int NOT NULL AUTO_INCREMENT,
```

```
  idSucursal int NOT NULL,
  departamento varchar(100) NOT NULL,
  localidad varchar(100) NOT NULL,
  callePrincipal varchar(150) NOT NULL,
  numero varchar(20),
  calleInterseccion varchar(150),
  observacion varchar(255),
  PRIMARY KEY (idDireccionSuc),
  KEY fk_dir_sucursal (idSucursal),
  CONSTRAINT fk_dir_sucursal
    FOREIGN KEY (idSucursal)
    REFERENCES sucursal (idSucursal)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ==============================
-- TABLA DISPONIBILIDAD
-- ==============================
CREATE TABLE disponibilidad (
  idProducto int NOT NULL,
  idSucursal int NOT NULL,
  cantidad int NOT NULL,
  PRIMARY KEY (idProducto,idSucursal),
  KEY fk_disp_sucursal (idSucursal),
  CONSTRAINT fk_disp_producto
    FOREIGN KEY (idProducto)
    REFERENCES producto (idProducto),
  CONSTRAINT fk_disp_sucursal
    FOREIGN KEY (idSucursal)
    REFERENCES sucursal (idSucursal)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- ================================
-- TABLA REGISTRO COSTO
-- ================================
CREATE TABLE registro_costo (
  idRegistroCosto int NOT NULL AUTO_INCREMENT,
  idProducto int NOT NULL,
  idProveedor int NOT NULL,
  costo decimal(10,2) NOT NULL,
  fechaHora datetime NOT NULL,
  porcentajeIVA decimal(5,2) NOT NULL,
  PRIMARY KEY (idRegistroCosto),
  KEY fk_rc_producto (idProducto),
  KEY fk_rc_proveedor (idProveedor),
  CONSTRAINT fk_rc_producto
    FOREIGN KEY (idProducto)
    REFERENCES producto (idProducto),
  CONSTRAINT fk_rc_proveedor
    FOREIGN KEY (idProveedor)
    REFERENCES proveedor (idProveedor)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- ================================
-- TABLA MOVIMIENTO STOCK
-- ================================
CREATE TABLE movimiento_stock (
  idMovimientoStock int NOT NULL AUTO_INCREMENT,
  idProducto int NOT NULL,
  idSucursal int NOT NULL,
  idUsuario int NOT NULL,
  tipoMovimiento varchar(50) NOT NULL,
  cantidad int NOT NULL,
```

```
  fechaHora datetime NOT NULL,
  motivo varchar(255),
  PRIMARY KEY (idMovimientoStock),
  KEY fk_ms_producto (idProducto),
  KEY fk_ms_sucursal (idSucursal),
  KEY fk_ms_usuario (idUsuario),
  CONSTRAINT fk_ms_producto
    FOREIGN KEY (idProducto)
    REFERENCES producto (idProducto),
  CONSTRAINT fk_ms_sucursal
    FOREIGN KEY (idSucursal)
    REFERENCES sucursal (idSucursal),
  CONSTRAINT fk_ms_usuario
    FOREIGN KEY (idUsuario)
    REFERENCES usuario (idUsuario)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
SET FOREIGN_KEY_CHECKS = 1;
```

Alcance del despliegue realizado

Es importante señalar que el uso de Render y Railway en este proyecto se realizó exclusivamente como entorno de prueba de despliegue y validación técnica.

En un escenario real de implementación empresarial, se espera que el sistema sea desplegado en un servicio de hosting profesional o infraestructura administrada por la empresa, que garantice:

- mayor control sobre la infraestructura
- políticas de seguridad adecuadas
- respaldo y monitoreo de la base de datos
- disponibilidad y estabilidad del servicio

De esta forma, las pruebas realizadas en estos servicios permitieron validar la portabilidad del sistema y la viabilidad técnica de su despliegue en la nube, constituyendo una etapa previa a una posible implementación en un entorno productivo.

Ajustes necesarios para la conexión entre Railway y DBeaver

Durante las pruebas de despliegue se utilizó DBeaver como herramienta de administración de la base de datos MySQL alojada en Railway. Para establecer correctamente la conexión fue necesario realizar algunos ajustes en la configuración del cliente de base de datos.

Configuración del puerto de conexión

A diferencia del entorno local de desarrollo, donde MySQL suele ejecutarse en el puerto estándar 3306, la base de datos proporcionada por Railway utiliza un puerto dinámico asignado por la plataforma.

Por esta razón fue necesario utilizar el puerto indicado en las variables de conexión del proyecto Railway y configurarlo manualmente en la conexión creada en DBeaver.

Uso de credenciales generadas por Railway

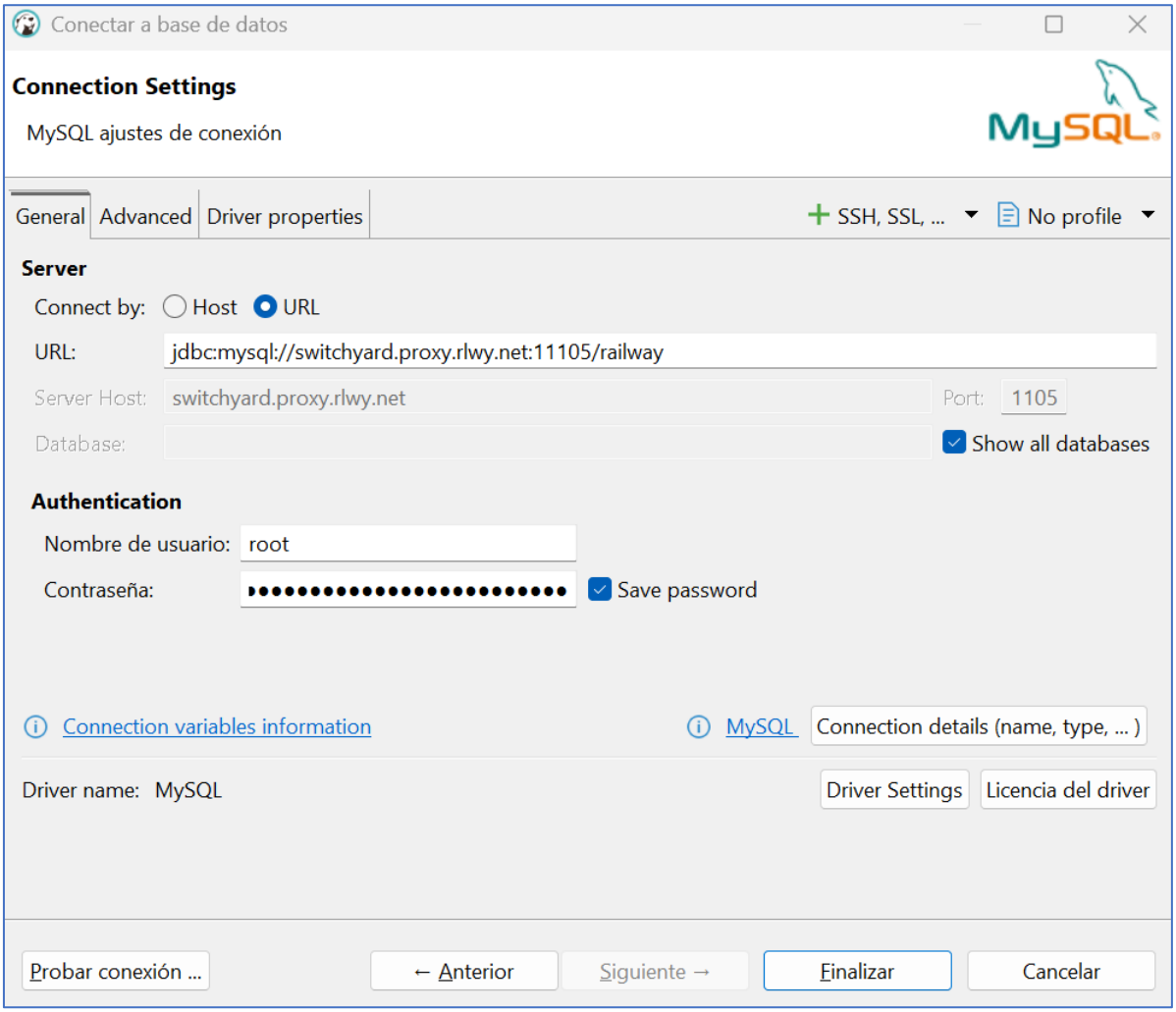
Las credenciales de acceso (host, usuario, contraseña y nombre de base de datos) fueron obtenidas desde el panel de configuración del servicio MySQL en Railway. Estos datos fueron utilizados para configurar la conexión en DBeaver.

Los parámetros principales configurados fueron:

- Host: proporcionado por Railway
- Port: puerto asignado por Railway
- Database: nombre de la base creada automáticamente por el servicio
- Username: usuario generado por Railway
- Password: contraseña generada por Railway
- Verificación de conexión remota

Una vez configurados estos parámetros en DBeaver, se realizó una prueba de conexión que permitió acceder a la base de datos remota, visualizar las tablas creadas y ejecutar consultas SQL para verificar el funcionamiento del sistema.

Esto permitió administrar la base de datos alojada en Railway de forma similar a un entorno local, facilitando la inspección de tablas, ejecución de scripts SQL y validación de los datos generados por la API del sistema.


Conectar a base de datos
Connection Settings
MySQL ajustes de conexión
MySQL
General
Advanced
Driver properties
SSH, SSL, ...
No profile
Server
Connect by:
Host
URL
URL:
jdbc:mysql://switchyard.proxy.rlwy.net:11105/railway
Server Host:
switchyard.proxy.rlwy.net
Port:
1105
Database:
Show all databases
Authentication
Nombre de usuario:
root
Contraseña:
Save password
Connection variables information
MySQL
Connection details (name, type, ...)
Driver name: MySQL
Driver Settings
Licencia del driver
Probar conexión ...
← Anterior
Siguiente →
Finalizar
Cancelar

Anexo F – Repositorio del sistema

El código fuente correspondiente al sistema Multiserv Stock (versión 1.0) se encuentra disponible en el siguiente repositorio:

https://github.com/enzofalcon/multiserv-stock

El repositorio incluye la estructura completa del proyecto, abarcando tanto el frontend como el backend, así como los archivos de configuración necesarios para su ejecución en entorno local y mediante contenedores Docker.

Asimismo, se dispone de los scripts de base de datos, permitiendo la correcta inicialización del sistema y su posterior utilización en distintos entornos de despliegue.

El acceso al repositorio tiene como objetivo facilitar la revisión técnica del sistema, así como su posible reutilización, mantenimiento o evolución futura.

Anexo G – Glosario de términos

Conceptos funcionales del sistema

Producto:

Entidad que representa un artículo gestionado por el sistema. Contiene información como descripción y stock mínimo.

Proveedor:

Entidad que representa a la empresa o persona que suministra productos. Puede estar activo o inactivo dentro del sistema.

Sucursal:

Unidad física o punto de almacenamiento donde se gestionan productos. Cada sucursal posee su propio stock.

Stock:

Cantidad disponible de un producto en una sucursal determinada.

Disponibilidad:

Registro que relaciona un producto con una sucursal, indicando la cantidad disponible del mismo.

Stock mínimo:

Valor definido para cada producto que indica el umbral mínimo recomendado de existencia.

Registro de costo:

Registro histórico del costo de un producto asociado a un proveedor, incluyendo el porcentaje de IVA correspondiente.

IVA (Impuesto al Valor Agregado):

Porcentaje aplicado al costo de un producto según normativa vigente.

Usuario:

Entidad que representa a una persona con acceso al sistema. En la versión actual, su gestión no es administrable desde la interfaz.

Sesión:

Estado que indica que un usuario ha iniciado sesión en el sistema y se encuentra autenticado.

Autenticación:

Proceso mediante el cual el sistema verifica la identidad de un usuario a través de sus credenciales.

Dashboard:

Pantalla principal que muestra información resumida del sistema, como cantidad de productos y alertas de stock.

Conceptos técnicos y de desarrollo

ABM (Alta, Baja, Modificación):

Conjunto de operaciones básicas para la gestión de entidades dentro del sistema.

CRUD:

Sigla que representa las operaciones básicas: Crear, Leer, Actualizar y Eliminar.

API (Interfaz de Programación de Aplicaciones):

Conjunto de endpoints que permiten la comunicación entre el frontend y el backend.

API REST:

Estilo de arquitectura basado en el uso de métodos HTTP como GET, POST y PUT.

Endpoint:

Ruta específica de la API que permite realizar una operación.

Backend:

Parte del sistema encargada de la lógica de negocio y acceso a datos.

Frontend:

Interfaz visual con la que interactúa el usuario.

DOM (Document Object Model):

Estructura del documento HTML que puede ser manipulada mediante JavaScript.

Fetch:

Función de JavaScript utilizada para realizar solicitudes HTTP.

JSON (JavaScript Object Notation):

Formato de intercambio de datos entre cliente y servidor.

JSON.parse:

Método que convierte un texto en formato JSON a un objeto JavaScript.

HTTP (HyperText Transfer Protocol):

Protocolo utilizado para la comunicación entre cliente y servidor.

GET / POST / PUT:

Métodos HTTP utilizados para interactuar con la API.

Submit:

Evento que se dispara al enviar un formulario.

Truthy / Falsy:

Valores que en JavaScript se interpretan como verdaderos o falsos en expresiones booleanas.

Arquitectura y diseño del sistema

Arquitectura DAO (Data Access Object):

Patrón que separa la lógica de acceso a datos del resto de la aplicación.

PDO (PHP Data Objects):

Extensión de PHP para acceso seguro a bases de datos mediante consultas preparadas.

conn: PDO:

Referencia a la conexión activa a la base de datos utilizando PDO.

Consultas preparadas:

Mecanismo que permite ejecutar consultas SQL de forma segura, evitando inyecciones SQL.

Hardcodeado:

Valor o configuración definida directamente en el código fuente.

Flujo de datos:

Secuencia de interacción entre componentes del sistema (por ejemplo: Vista → JS → API → Base de datos).

Workflow:

Flujo de trabajo que describe las etapas de un proceso dentro del sistema.

Wireframe:

Representación esquemática de una interfaz en etapas de diseño.

UX (User Experience):

Experiencia del usuario al interactuar con el sistema.

UI (User Interface):

Diseño visual de la interfaz.

Layout:

Estructura visual de los elementos en pantalla.

Breadcrumb:

Elemento de navegación que indica la ubicación actual dentro del sistema.

Infraestructura y despliegue

Base de datos:

Sistema de almacenamiento estructurado donde se persiste la información.

Servidor VPS (Virtual Private Server):

Servidor virtual utilizado para alojar aplicaciones.

Docker:

Herramienta que permite ejecutar aplicaciones en contenedores.

Contenedor:

Entorno aislado que incluye todo lo necesario para ejecutar una aplicación.

Deploy (Despliegue):

Proceso de publicación de una aplicación en un entorno productivo.

PaaS (Platform as a Service):

Plataforma que permite desplegar aplicaciones sin gestionar la infraestructura subyacente (ej.: Render).

Seguridad

Hash:

Proceso de transformación de datos en un formato cifrado no reversible.

bcrypt / Argon2:

Algoritmos de hashing utilizados para proteger contraseñas.

Seguridad de login:

Conjunto de prácticas para proteger el acceso al sistema (hash, validación, protección de endpoints).

Protección de endpoints:

Mecanismo que restringe el acceso a la API solo a usuarios autenticados.

Token:

Elemento utilizado para validar autenticación en sistemas más avanzados (no implementado en esta versión).

Otros conceptos utilizados

Backoffice:

Sección interna del sistema utilizada para la gestión administrativa.

localStorage:

Almacenamiento en el navegador del cliente.

Cross-path / rutas absolutas:

Forma de definir rutas completas para acceder a recursos dentro del sistema.

SPA (Single Page Application):

Aplicación de una sola página. No utilizada en este sistema, que emplea vistas independientes.

Flujo “fake”:

Simulación de comportamiento sin persistencia real, utilizada en etapas iniciales de desarrollo.